المرجع الاحترافي الشامل

JSON & YAML

من الصفر إلى الاحتراف (Expert Level)

دليل عملي متكامل لفهم صيغ تبادل البيانات والإعداد

من الأساسيات إلى الإنتاج: Python • Backend • MLOps • DevOps • Cloud

جدول المحتويات

جدول المحتويات 2
الفصل الأول — مقدمة شاملة: JSON وYAML 9
أولًا: JSON — JavaScript Object Notation 9
ما هو JSON؟ 9
ليه ظهرت JSON؟ وإيه المشكلة اللي جات تحلها؟ 9
تاريخ JSON وتطورها 9
فلسفة تصميم JSON 10
المواصفات الرسمية (Specification) 10
أهم استخداماتها 10
أهم الشركات والمشاريع المعتمدة عليها 11
مميزات JSON 11
عيوب JSON 11
القيود (Limitations) 11
متى تستخدم JSON؟ 11
متى لا تستخدم JSON؟ 11
أشهر بدائل JSON 12
ثانيًا: YAML — YAML Ain't Markup Language 13
ما هي YAML؟ 13
ليه ظهرت YAML؟ وإيه المشكلة اللي جات تحلها؟ 13
تاريخ YAML وتطورها 13
فلسفة تصميم YAML 13
المواصفات الرسمية (Specification) 14
أهم استخداماتها 14
أهم الشركات والمشاريع المعتمدة عليها 14
مميزات YAML 14
عيوب YAML 14
القيود (Limitations) 15
متى تستخدم YAML؟ 15
متى لا تستخدم YAML؟ 15
أشهر بدائل YAML 15
لمحة سريعة: JSON مقابل YAML 16
الأسئلة الشائعة 17
أخطاء شائعة عند المبتدئين 17
أفضل الممارسات 17
ملخص الفصل 17
تمارين عملية 18

الفصل الثاني — البنية — Syntax & Structure 19
أولًا: بنية JSON 19
القواعد الأساسية (Syntax Rules) 19
Objects / Mappings 19
Arrays / Lists 19
Strings 19
Numbers 20
Null وBoolean 20
Nested Lists وNested Objects 20
Unicode وEscaping 20
حالات حافة (Edge Cases) في الأرقام 21
Flow Style في YAML بالتفصيل 21
ثانيًا: بنية YAML 22
القواعد الأساسية 22
Mappings (المكافئ لـ Objects) 22
Lists (المكافئ لـ Arrays) 22
Strings 22
Numbers, Boolean, Null 22
Nested Lists وNested Objects 23
Multi-line Strings 23
Documents متعددة في ملف واحد 23
Special Characters وFormatting 24
مقارنة سريعة: بنية JSON مقابل بنية YAML 25
الأسئلة الشائعة 26
أخطاء شائعة 26
أفضل الممارسات 26
ملخص الفصل 26
تمارين عملية 26
الفصل الثالث — Core Concepts — المفاهيم الأساسية 28
Serialization وDeserialization 28
Parsing 28
Encoding وDecoding 28
Hierarchical Data وStructured Data وNested Data 28
مفاهيم خاصة بـ YAML: Anchors وAliases وTags وMerge Keys 29
Anchors (&) وAliases (*) 29
Merge Keys (<<) 29
Tags 29
Human Readability مقابل Machine Readability 30

كيف تعمل الـ Parsers داخليًا (High-Level) 31
الأسئلة الشائعة 31
أخطاء شائعة 31
أفضل الممارسات 32
ملخص الفصل 32
تمارين عملية 32
الفصل الرابع — التعامل مع Python 33
أولًا: JSON في Python — وحدة json 33
json.loads() و json.load() 33
json.dumps() و json.dump() 33
Custom Encoder و Custom Decoder 34
ثانيًا: YAML في Python 35
PyYAML 35
ruamel.yaml 35
Custom Constructors و Custom Representers 36
مقارنة: PyYAML مقابل ruamel.yaml 37
الأسئلة الشائعة 37
أخطاء شائعة 37
أفضل الممارسات 37
ملخص الفصل 38
تمارين عملية 38
الفصل الخامس — Validation & Schema — التحقق من صحة البيانات 39
أولًا: JSON Schema 39
Properties وRequired 39
Pattern وEnum 39
OneOf, AnyOf, AllOf 39
Nested Validation وCustom Validation 40
ثانيًا: التحقق من صحة YAML 41
طرق التحقق الشائعة 41
Common Practices 41
الأسئلة الشائعة 42
أخطاء شائعة 42
أفضل الممارسات 42
ملخص الفصل 42
تمارين عملية 42
الفصل السادس — الاستخدامات العملية 44
REST APIs وGraphQL 44
Backend وFrontend Development 44

Data Science وMachine Learning وPython Applications 44
Docker Compose وDocker وMLOps 45
Kubernetes 45
CI/CD وGitHub Actions 45
Environment Configuration وCloud Platforms 47
IDE Configuration وVS Code Settings وLogging 47
Data Exchange وMicroservices 47
الأسئلة الشائعة 47
أخطاء شائعة 47
أفضل الممارسات 48
ملخص الفصل 48
تمارين عملية 48
الفصل السابع — الأداء — Performance 49
Parsing Performance 49
Serialization Speed 49
File Size 49
Memory Usage 49
Compression 49
Streaming 49
Optimization Techniques 50
مقارنة الأداء: JSON مقابل YAML 51
مثال عملي: قياس الفرق بنفسك (Benchmark بسيط) 51
الأسئلة الشائعة 51
أخطاء شائعة 52
أفضل الممارسات 52
ملخص الفصل 52
تمارين عملية 52
الفصل الثامن — Debugging — تتبع وحل الأخطاء 53
Parsing Errors في JSON 53
أشهر أخطاء JSON 53
Parsing Errors في YAML 54
Indentation Problems 54
Encoding Issues 54
Validation Errors 54
أدوات (Debugging Tools) Debugging 55
استراتيجية عامة لحل مشاكل Parsing 55
الأسئلة الشائعة 55
أخطاء شائعة 55

أفضل الممارسات 55
ملخص الفصل 56
تمارين عملية 56
الفصل التاسع — Security — الأمان 57
المبدأ العام — Secure Parsing 57
خطيرة لو استخدمتها غلط — Unsafe Loaders ليه YAML 57
Trusted مقابل Untrusted Data 57
الحماية الصحيحة 58
نظرة عامة على طبيعة المشكلة — YAML Exploits وCode Injection 58
أنواع عامة — Common Vulnerabilities 58
الحماية من استنزاف الموارد (Resource Exhaustion) 58
ملخص عملي — Best Security Practices 60
الأسئلة الشائعة 60
أخطاء شائعة 60
أفضل الممارسات 60
ملخص الفصل 61
تمارين عملية 61
الفصل العاشر — Best Practices — أفضل الممارسات 62
Naming Convention 62
File Organization 62
Maintainability وReadability 62
Documentation 62
Version Control 63
Reusability 63
Formatting Standards 63
قائمة مراجعة سريعة (Checklist) قبل أي Commit 63
الأسئلة الشائعة 63
أخطاء شائعة 64
أفضل الممارسات — ملخص شامل 64
ملخص الفصل 64
تمارين عملية 64
الفصل الحادي عشر — مشاريع عملية 65
مشاريع JSON 65
مشروع ١: إعداد ملف Configuration لتطبيق Backend 65
مشروع ٢: تحويل CSV إلى JSON 65
مشروع ٣: تحويل JSON إلى CSV 65
مشروع ٤: إعداد ملفات Localization (ترجمة الواجهات) 66
مشروع ٥: إعداد REST API Response موحّد 66

مشروع ٦: تحويل JSON إلى Excel 66
مشروع ٧: تخزين بيانات تطبيق محلي (Local App Data Storage) 67
مشروع ٨: ملفات Settings منفصلة لكل بيئة 67
مشاريع YAML 69
مشروع ٩: Docker Compose لتطبيق ويب كامل 69
مشروع ١٠: Kubernetes Manifest بسيط لخدمة API 69
مشروع ١١: GitHub Actions Workflow لاختبار وبناء مشروع Python 70
مشروع ١٢: ملف إعداد Machine Learning (Hyperparameters) 70
مشروع ١٣: Logging Configuration 70
مشروع ١٤: CI/CD Pipeline مستقل (GitLab CI) 71
مشروع ١٥: Python Project Configuration (mkdocs.yml) 71
مشروع ١٦: Application Settings شامل لتطبيق Backend 71
أفضل الممارسات 72
ملخص الفصل 72
تمارين عملية 72
الفصل الثاني عشر — المقارنات 74
JSON مقابل YAML 74
JSON مقابل XML 75
JSON مقابل TOML 75
YAML مقابل TOML 75
YAML مقابل INI 76
YAML مقابل ملفات ENV 76
جدول القرار السريع: أي صيغة أختار؟ 76
الأسئلة الشائعة 77
أخطاء شائعة 77
أفضل الممارسات 77
ملخص الفصل 77
تمارين عملية 77
الفصل الثالث عشر — Professional Workflow 78
دورة حياة ملفات الإعداد (Configuration Lifecycle) 78
تنظيم ملفات Configuration في المشاريع الكبيرة 78
إدارة البيئات المختلفة (Development / Testing / Production) 80
إدارة الإعدادات الحساسة (Secrets Management) 80
كيف يستخدم كل تخصص هذه الصيغ فعليًا 81
Backend Engineer 81
Machine Learning Engineer 81
DevOps Engineer 81
الأسئلة الشائعة 81

أخطاء شائعة 81
أفضل الممارسات 81
ملخص الفصل 81
تمارين عملية 82
المراجع الرسمية ..... **خطأ! الإشارة المرجعية غير معرّفة.**

الفصل الأول — مقدمة شاملة: JSON وYAML

قبل ما ندخل في أي تفصيلة تقنية، لازم نفهم حاجة أساسية جدًا: JSON وYAML مش مجرد "صيغ لكتابة البيانات"، دول في الحقيقة حلول لمشكلة إنسانية قديمة قوي، وهي: إزاي أخلي جهازين مختلفين (أو برنامجين، أو إنسان وجهاز) يتفاهموا على شكل موحّد للبيانات من غير لبس؟

الكتاب ده هيبني عندك فهم عميق مش سطحي؛ مش هنكتفي بـ"إزاي تكتب { } و[ ]"، هنفهم مع بعض ليه الصيغتين اتصمموا بالشكل ده، وإيه اللي كان بيحصل قبلهم، وإيه المشاكل اللي حلّوها، عشان لما تفتح أي ملف Config في أي مشروع على GitHub تقدر تقرأه وتفهم قرار المهندس اللي كتبه.

أولًا: JSON — JavaScript Object Notation

ما هو JSON؟

JSON هي اختصار لـ JavaScript Object Notation، وهي صيغة نصية خفيفة لتمثيل البيانات المهيكلة (Structured Data)، معتمدة بالكامل على نوعين بس من البُنى: الكائنات (Objects) اللي هي مجموعة أزواج مفتاح-قيمة، والمصفوفات (Arrays) اللي هي قوائم مرتبة من القيم.

الاسم بيقول "JavaScript" لكن الحقيقة إن JSON مستقلة تمامًا عن أي لغة برمجة؛ هي مجرد Notation (طريقة كتابة) استُلهمت من الطريقة اللي بيتكتب بيها الـ Object Literals في JavaScript، لكن أي لغة برمجة النهاردة — Python وJava وC# وGo وRust وPHP — عندها Parser خاص بيها يقرأ ويكتب JSON من غير أي علاقة بـ JavaScript نفسها.

ليه ظهرت JSON؟ وإيه المشكلة اللي جات تحلها؟

في أواخر التسعينات وبداية الألفينات، كان الويب بيتحول من صفحات ثابتة لتطبيقات تفاعلية (Web 2.0). المتصفح بقى محتاج يكلم السيرفر ويجيب بيانات جديدة من غير ما يعمل Reload لكل الصفحة — ده اللي عُرف بعدين بـ AJAX (Asynchronous JavaScript and XML).

المشكلة إن الصيغة المستخدمة وقتها كانت XML، وXML قوي وغني بالمواصفات، لكنه Verbose جدًا — يعني كتير من الكلام الزيادة (Tags مفتوحة ومقفولة، Attributes، Namespaces) عشان توصّل معلومة بسيطة. Douglas Crockford، وهو مهندس في شركة State Software، لاحظ إن المطورين بيحتاجوا صيغة أخف وأسرع تتحول مباشرة لكائن JavaScript جاهز للاستخدام في المتصفح من غير خطوة Parsing معقدة.

سنة ٢٠٠١ قدّم Crockford صيغة JSON رسميًا، ومهم نعرف حاجة لطيفة: هو ما "اخترعش" حاجة جديدة كليًا، هو "اكتشف" إن الطريقة اللي JavaScript بيمثّل بيها الكائنات أصلًا (Object Literals) صالحة تمامًا كصيغة تبادل بيانات مستقلة، فبس وثّقها ونظّمها في مواصفة (Specification) بسيطة جدًا — الصفحة اللي بتشرح كل قواعد JSON ممكن تتكتب في أقل من صفحة واحدة.

> **ملاحظة**
>
> الدرس المهم هنا: أحيانًا أفضل الحلول الهندسية مش اختراع حاجة جديدة، لكن ملاحظة نمط موجود فعلًا وتوثيقه وتبسيطه. JSON هي مثال كلاسيكي على مبدأ "البساطة المتعمدة" (Deliberate Simplicity) في هندسة البرمجيات.

تاريخ JSON وتطورها

- ٢٠٠١: Douglas Crockford يوثّق ويعلن عن JSON، ويسجّل الدومين json.org.
- ٢٠٠٥: ياهو تعلن دعمها الرسمي لـ JSON كصيغة بديلة عن XML في الـ Web Services بتاعتها.
- ٢٠٠٦: Google تبدأ تستخدم JSON بشكل موسّع في خدماتها.
- ٢٠١٣: JSON تتحول لمعيار دولي رسمي: ECMA-404.
- ٢٠١٤ وما بعدها: IETF تصدر RFC 7159 ثم RFC 8259 (2017) كمواصفة رسمية نهائية معتمدة عالميًا للإنترنت.
- النهاردة: JSON هي الصيغة الافتراضية لأي REST API تقريبًا، ولأغلب أنظمة الـ NoSQL زي MongoDB، ولملفات الإعداد في مشاريع Node.js وPython وغيرها.

فلسفة تصميم JSON

لو هتفهم JSON فهم عميق، لازم تستوعب المبادئ اللي بُنيت عليها الصيغة دي، لأن كل قرار تصميمي فيها نابع من فلسفة واضحة:

- البساطة قبل القوة (Simplicity over Power): JSON عن قصد ما فيهاش Comments، ما فيهاش أنواع بيانات معقدة زي Dates أو Binary، عشان تفضل الصيغة بسيطة وسريعة الفهم والتنفيذ في أي لغة.
- عدد أنواع بيانات محدود جدًا: بس ٦ أنواع (Object, Array, String, Number, Boolean, Null) — مفيش تعقيد زيادة.
- سهولة التحويل الآلي (Machine-Friendly): بنية JSON مصمّمة عشان الـ Parser يقدر يحوّلها مباشرة لهياكل بيانات في الذاكرة (Objects/Dictionaries وArrays/Lists) من غير خطوات وسيطة معقدة.
- قابلية القراءة للإنسان (Human-Readable) لكن مش على حساب سرعة الآلة: على عكس XML، JSON بتوازن بين وضوح النص للإنسان وسرعة تحليله للآلة.
- عدم الاعتماد على لغة معينة (Language-Independent): رغم الاسم، الصيغة مصممة عشان أي لغة تقدر تمثّلها بسهولة.

المواصفات الرسمية (Specification)

JSON موثّقة رسميًا في أكتر من مكان متوافقين مع بعض:

- ECMA-404: المعيار الرسمي الصادر عن منظمة Ecma International، بيوصف الـ Syntax فقط بدون دلالات إضافية.
- RFC 8259 (IETF): المعيار المعتمد للإنترنت، بيوضح تفاصيل زي الترميز (Encoding) وكيفية التعامل مع الأرقام الكبيرة والتكرار في المفاتيح.
- json.org: الموقع الأصلي اللي وضعه Crockford، فيه Diagram بصري بسيط جدًا (Railroad Diagram) بيشرح الـ Grammar بالكامل.

أهم استخداماتها

- REST APIs: الصيغة الافتراضية لطلبات واستجابات الـ APIs في كل مكان.
- ملفات الإعداد: package.json في Node.js، composer.json في PHP، tsconfig.json في TypeScript.
- قواعد بيانات NoSQL: MongoDB بيخزّن البيانات فعليًا كـ BSON، وهي نسخة ثنائية من JSON.
- تبادل البيانات بين الخدمات المصغّرة (Microservices).

- تخزين الإعدادات والـ State في تطبيقات الويب والموبايل.
- ملفات الـ Localization (ترجمة الواجهات).

أهم الشركات والمشاريع المعتمدة عليها

- Google: كل تقريبًا الـ APIs الخاصة بيها (Maps, YouTube Data API) بترجع JSON.
- GitHub، Facebook، Twitter/X: كل الـ Public APIs بتاعتهم أساسها JSON.
- Node.js Ecosystem بالكامل: npm نفسها مبنية حول package.json.
- MongoDB وElasticsearch: مبنيين على تمثيل شبيه بـ JSON للبيانات.

مميزات JSON

- بساطة شديدة في القواعد — تتعلمها في دقايق.
- دعم مدمج في كل لغات البرمجة الحديثة تقريبًا.
- حجم أصغر من XML لنفس البيانات.
- سرعة Parsing عالية جدًا مقارنة بـ XML.
- مناسبة جدًا لتبادل البيانات عبر الشبكة (Network).

عيوب JSON

- مفيش Comments، وده بيبقى مزعج في ملفات الإعداد الكبيرة اللي محتاجة توثيق.
- مفيش دعم مباشر لأنواع بيانات زي Date أو Set أو Binary — لازم تتحول لـ String أو Number يدويًا.
- مفيش Trailing Commas (فاصلة بعد آخر عنصر) — بتسبب أخطاء شائعة.
- صعبة القراءة شوية في الملفات الكبيرة جدًا مقارنة بـ YAML بسبب كثرة الأقواس والفواصل.
- مفيش دعم لـ References أو Anchors زي YAML، يعني لو نفس القيمة اتكررت لازم تتكرر كتابتها بالكامل.

القيود (Limitations)

- الأرقام: مفيش تفرقة بين الأعداد الصحيحة (Integers) والعشرية (Floats) في المواصفة نفسها، والتعامل مع الأرقام الكبيرة جدًا (زي ٦٤-bit integers) بيختلف من لغة للتانية.
- لا يوجد نوع بيانات للتاريخ (Date/Time) — دايمًا بيتحول لـ String بصيغة ISO 8601 بالاتفاق بين المطورين، مش بإلزام من المواصفة.
- لا يدعم التعليقات (Comments) رسميًا في أي Parser معياري.

متى تستخدم JSON؟

- لما تحتاج تبادل بيانات سريع بين Backend وFrontend عبر API.
- لما الملف هيتقرأ بواسطة آلة أكتر ما هيتعدّل يدويًا بواسطة إنسان بشكل متكرر.
- لما الأداء (Parsing Speed) وحجم البيانات (Payload Size) مهمين.
- لما محتاج توافق كامل مع كل بيئات التشغيل (Cross-platform Compatibility).

متى لا تستخدم JSON؟

- لما الملف محتاج تعليقات كتير لتوضيح كل إعداد (استخدم YAML أو TOML بدلًا منه).
- لما الملف طويل جدًا ومحتاج قراءة بشرية متكررة (YAML بيبقى أوضح بصريًا).
- لما محتاج مرجعية داخلية بين القيم YAML — (Anchors/References) بيدعم ده بشكل مباشر.

أشهر بدائل JSON

- XML: أقدم وأكثر Verbose، لكن أقوى في التحقق من الصحة (Validation) عبر XSD.
- YAML: أوضح للقراءة البشرية، بيُستخدم غالبًا في ملفات الإعداد.
- TOML: بسيط جدًا ومصمم خصيصًا لملفات الإعداد (Configuration)، بيُستخدم كتير في مشاريع Rust و(pyproject.toml) Python.
- Protocol Buffers / MessagePack: صيغ ثنائية (Binary) أسرع وأصغر حجمًا، لكن مش قابلة للقراءة البشرية مباشرة.

ثانيًا: YAML — YAML Ain't Markup Language

ما هي YAML؟

YAML اسمها اختصار ذاتي المرجع (Recursive Acronym) معناه "YAML Ain't Markup Language"، وده بحد ذاته بيوضح فلسفتها: هي مش لغة توصيف (Markup) زي HTML أو XML، هي لغة تسلسل بيانات (Data Serialization Language) مصممة أساسًا عشان تكون سهلة القراءة للإنسان قبل أي حاجة تانية.

الاسم القديم كان "Yet Another Markup Language"، لكن المطورين غيّروه لاحقًا لـ "YAML Ain't Markup Language" عشان يأكدوا إن التركيز مش على الـ Markup لكن على تمثيل البيانات (Data Representation).

ليه ظهرت YAML؟ وإيه المشكلة اللي جات تحلها؟

سنة ٢٠٠١، في نفس الفترة تقريبًا اللي ظهرت فيها JSON، كان في مجتمع Perl (خصوصًا Clark Evans) شايف إن الصيغ الموجودة وقتها (XML وحتى JSON اللي كانت لسه ناشئة) بتحتاج مجهود بصري كبير عشان الإنسان يقرأها بسرعة، خصوصًا في ملفات الإعداد الكبيرة اللي المبرمج بيرجعلها يعدّل فيها يدويًا كل شوية.

الفكرة الأساسية كانت: نعتمد على المسافات البادئة (Indentation) بدل الأقواس، ونستخدم بنية شبه لغة طبيعية (زي الـ To-Do Lists اللي بيكتبها أي حد بخط اليد) عشان يبقى الملف قريب من طريقة تفكير الإنسان مش الآلة، مع الاحتفاظ بالقدرة على تمثيل بيانات معقدة زي JSON بالظبط.

> **ملاحظة**
> نقطة مهمة جدًا: YAML هي فعليًا Superset لـ JSON — يعني أي ملف JSON صحيح هو تلقائيًا ملف YAML صحيح (بداية من YAML 1.2). ده معناه إن أي YAML Parser المفروض يقدر يقرأ JSON من غير مشاكل.

تاريخ YAML وتطورها

- ٢٠٠١: Clark Evans يبدأ المشروع بمساعدة Ingy döt Net وOren Ben-Kiki.
- ٢٠٠٤: إصدار YAML 1.0 الرسمي.
- ٢٠٠٥: YAML 1.1، وفيها مشاكل شهيرة زي تفسير القيمة NO كـ Boolean بدل String (مشكلة "Norway Problem").
- ٢٠٠٩: YAML 1.2، وفيها تم حل مشكلة التوافق مع JSON بشكل كامل، وأصبحت JSON فعليًا Subset منها.
- بعد كده: انتشار واسع جدًا مع صعود أدوات DevOps: Ansible وDocker Compose وKubernetes وGitHub Actions، كلها اختارت YAML كصيغة إعداد أساسية.

فلسفة تصميم YAML

- القراءة البشرية أولًا (Human-First Readability): البنية بتعتمد على المسافات والتنسيق البصري بدل الرموز، فبتقرب من شكل النص العادي.
- لا تكرار في الكتابة (DRY): عن طريق الـ Anchors (&) والـ Aliases (*)، تقدر تعرّف قيمة مرة واحدة وتستخدمها في أكتر من مكان.
- التوافق الكامل مع JSON: أي حاجة تقدر تعبّر عنها بـ JSON تقدر تعبّر عنها بـ YAML، والعكس مش صحيح دايمًا.
- المرونة في التمثيل: تقدر تكتب نفس البيانات بأكتر من شكل (Block Style أو Flow Style) حسب احتياجك.

- دعم التعليقات (#) بشكل أساسي في اللغة، على عكس JSON.

المواصفات الرسمية (Specification)

- YAML 1.2.2 هي أحدث نسخة رسمية من المواصفة (٢٠٢١)، متاحة بالكامل في yaml.org.
- المواصفة معقدة أكتر بكتير من JSON — ده سبب رئيسي في وجود اختلافات بسيطة بين الـ Parsers المختلفة (PyYAML وruamel.yaml وSnakeYAML وغيرهم).

أهم استخداماتها

- Docker Compose: تعريف الخدمات (Services) والشبكات والـ Volumes.
- Kubernetes: كل الـ Manifests (Deployments, Services, ConfigMaps) بتتكتب بـ YAML.
- CI/CD: GitHub Actions، GitLab CI، CircleCI كلها بتعتمد على YAML لتعريف الـ Pipelines.
- Ansible: كل الـ Playbooks مكتوبة بـ YAML بالكامل.
- ملفات إعداد تطبيقات Python: زي mkdocs.yml، وملفات إعداد نماذج الـ Machine Learning (Hydra, MLflow).

أهم الشركات والمشاريع المعتمدة عليها

- Google: مطورين أساسيين لـ Kubernetes، اللي كل إعداداته YAML.
- Red Hat: مطورين Ansible، أداة الأتمتة الأشهر المعتمدة كليًا على YAML.
- GitHub وGitLab: بنية الـ CI/CD بالكامل معتمدة على YAML.
- Netflix وUber: بيستخدموا YAML على نطاق واسع في إعدادات الـ Microservices والـ Infrastructure as Code.

مميزات YAML

- قابلية قراءة عالية جدًا للإنسان — الأقرب لطريقة تفكيرنا الطبيعية.
- دعم التعليقات، وده أساسي لأي ملف إعداد كبير.
- دعم Anchors وAliases لتقليل التكرار.
- مرونة عالية في تمثيل البيانات (Multi-line Strings, Multiple Documents في ملف واحد).
- توافق كامل مع JSON، يعني تقدر تستخدم أي أداة تفهم JSON كنقطة بداية.

عيوب YAML

- حساسة جدًا للمسافات (Indentation) — مسافة زيادة أو ناقصة ممكن تكسر الملف بالكامل أو الأسوأ تغيّر معناه من غير ما تظهر أي خطأ.
- مشكلة "Norway Problem": قيم زي NO أو YES أو ON أو OFF ممكن تتفسر كـ Boolean من غير قصد بدل ما تفضل String.
- تعقيد المواصفة أدّى لوجود اختلافات فعلية بين الـ Parsers المختلفة.
- خطر أمني حقيقي: بعض الـ Parsers (زي PyYAML القديم) بتقدر "تنفّذ كود" لو استخدمت Loader غير آمن — هنتكلم عن ده بالتفصيل في فصل الأمان.

- أبطأ في الـ Parsing مقارنة بـ JSON بسبب تعقيد قواعدها.

القيود (Limitations)

- عدم وجود معيار صارم واحد متفق عليه ١٠٠٪ بين كل التطبيقات، فبعض الميزات (زي Merge Keys <<) مش جزء رسمي من المواصفة لكنها اتفاق ضمني بين المجتمع.
- صعوبة تتبّع الأخطاء (Debugging) في الملفات الكبيرة جدًا بسبب الاعتماد على المسافات بدل الأقواس الصريحة.

متى تستخدم YAML؟

- ملفات الإعداد اللي هيقرأها ويعدّل فيها بني آدمين بشكل متكرر (Docker Compose, Kubernetes, CI/CD).
- لما محتاج تعليقات لتوضيح كل إعداد.
- لما عندك بيانات متكررة ومحتاج تقلل التكرار عن طريق Anchors.

متى لا تستخدم YAML؟

- في تبادل البيانات عبر الشبكة بين خدمات (استخدم JSON، أخف وأسرع وأكتر أمانًا).
- لما البيانات جايه من مصدر غير موثوق (Untrusted Source) بدون ضبط الـ Loader الآمن.
- لما الأداء (Performance) في الـ Parsing عامل حرج، زي ملايين الطلبات في الثانية.

أشهر بدائل YAML

- JSON: أسرع وأبسط، لكن أقل قابلية للقراءة في الملفات الكبيرة ولا يدعم تعليقات.
- TOML: أبسط من YAML، أقل عرضة للأخطاء بسبب المسافات، بيُستخدم في Rust (Cargo.toml) وPython (pyproject.toml).
- HCL (HashiCorp Configuration Language): مستخدم في Terraform، مصمم خصيصًا لوصف البنية التحتية.
- INI: بسيط جدًا لكنه محدود، بيُستخدم في إعدادات قديمة (Windows Config Files).

لمحة سريعة: JSON مقابل YAML

الجدول التالي بيدّيك نظرة سريعة قبل ما ندخل في التفاصيل العميقة في الفصول القادمة (وخصوصًا الفصل الثاني عشر اللي هيكون مخصص بالكامل للمقارنات التفصيلية):

المعيار	JSON	YAML
سهولة القراءة البشرية	متوسطة	عالية جدًا
سرعة الـ Parsing	عالية	أبطأ نسبيًا
دعم التعليقات	❌ لا يوجد	✅ يوجد (#)
الاعتماد على المسافات	❌ لا (أقواس صريحة)	✅ نعم (Indentation)
الاستخدام الشائع	APIs وتبادل البيانات	ملفات الإعداد وDevOps
دعم Anchors/References	❌ لا يوجد	✅ يوجد
تعقيد المواصفة	بسيطة جدًا (صفحة واحدة تقريبًا)	معقدة (عشرات الصفحات)
التوافق	مستقلة	Superset كامل لـ JSON (منذ ١,٢)

الأسئلة الشائعة

❓ هل JSON جزء من YAML ولا العكس؟

من نسخة YAML 1.2، أي ملف JSON صحيح هو تلقائيًا ملف YAML صحيح ١٠٠٪. يعني YAML هي الأشمل (Superset)، لكن العكس مش صحيح — YAML فيها ميزات (زي Anchors والتعليقات) مش موجودة في JSON.

❓ هل ينفع أستخدم JSON بدل YAML في Kubernetes أو Docker Compose؟

نعم فعليًا، لأن YAML بتقبل JSON كـ Subset منها، فتقدر تكتب ملف Kubernetes بصيغة JSON بالكامل وهيشتغل. لكن عمليًا الكل بيفضّل YAML في المشاريع دي بسبب التعليقات وسهولة القراءة.

❓ ليه في الأصل ظهرت صيغتين مختلفتين لحل نفس المشكلة تقريبًا؟

لأن الأولوية كانت مختلفة: JSON ظهرت من احتياج فني ضيق (تبادل بيانات في المتصفح بسرعة)، أما YAML ظهرت من احتياج مختلف (قراءة بشرية مريحة لملفات إعداد). الاختلاف في الأولوية أدّى لاختلاف في التصميم رغم تشابه الهدف العام.

أخطاء شائعة عند المبتدئين

⚠ خطأ شائع

الاعتقاد إن JSON مرتبطة حصريًا بـ JavaScript فقط، والتردد في استخدامها مع Python أو Java.

⚠ خطأ شائع

استخدام YAML لتبادل بيانات API بدل ملفات الإعداد، وده بيأثر سلبًا على الأداء من غير داعي.

⚠ خطأ شائع

الخلط بين نسخ YAML المختلفة (١,١ مقابل ١,٢) وعدم معرفة إن سلوك بعض القيم زي on/off بيختلف بينهم.

أفضل الممارسات

☑ أفضل الممارسات

استخدم JSON لأي بيانات هتتبادل بين الخدمات عبر الشبكة (APIs).

☑ أفضل الممارسات

استخدم YAML لملفات الإعداد اللي هيقرأها ويعدّلها بني آدمين بانتظام.

☑ أفضل الممارسات

لو مش متأكد، اسأل نفسك: مين هيقرأ الملف ده أكتر، إنسان ولا آلة؟ الإجابة بتحدد الاختيار الصح.

ملخص الفصل

- JSON صيغة بسيطة جدًا ظهرت سنة ٢٠٠١ عشان تحل مشكلة تبادل البيانات في تطبيقات الويب التفاعلية (AJAX)، وأصبحت المعيار العالمي لـ REST APIs.
- YAML ظهرت في نفس الفترة تقريبًا بفلسفة مختلفة: الأولوية للقراءة البشرية، وأصبحت المعيار الفعلي لملفات الإعداد في DevOps وKubernetes وDocker وCI/CD.
- منذ YAML 1.2، أي JSON صحيح هو YAML صحيح تلقائيًا — العلاقة بينهم Superset/Subset مش تنافس كامل.
- الاختيار بين الاتنين مبني على سؤال بسيط: هل الملف ده للآلة ولا للإنسان؟

تمارين عملية

تمرين ١: اكتب بالكلام (من غير كود) الفرق الجوهري بين فلسفة تصميم JSON وفلسفة تصميم YAML.

> **الحل المقترح**
>
> JSON صُمّمت للآلة أولًا مع الحفاظ على قابلية قراءة معقولة، أما YAML صُمّمت للإنسان أولًا مع الحفاظ على قدرة الآلة على تحليلها؛ الفرق في الأولوية هو أصل كل الاختلافات الأخرى بينهم.

تمرين ٢: ابحث عن ملف package.json في أي مشروع Node.js، وملف docker-compose.yml في أي مشروع Docker، وقارن بينهم: أيهما فيه تعليقات؟ أيهما أسهل تعديله يدويًا في رأيك؟

> **الحل المقترح**
>
> package.json عادة بدون تعليقات لأنه يُقرأ بواسطة npm أكثر من تعديل يدوي متكرر بالتفصيل، بينما docker-compose.yml غالبًا يحتوي تعليقات لأنه يُعدَّل باستمرار من قبل المطورين.

الفصل الثاني — البنية Syntax & Structure

في الفصل ده هنغوص في تفاصيل الـ Syntax نفسها لكل صيغة، مع أمثلة كتير جدًا، عشان تقدر تقرأ وتكتب أي ملف JSON أو YAML من غير ما ترجع للتوثيق كل شوية.

أولًا: بنية JSON

القواعد الأساسية (Syntax Rules)

- البيانات مكوّنة من أزواج مفتاح-قيمة داخل Objects، أو قوائم قيم داخل Arrays.
- المفاتيح (Keys) دايمًا لازم تكون String محاطة بعلامتي تنصيص مزدوجة " " — مش مسموح بعلامة تنصيص مفردة.
- القيم ممكن تكون: String, Number, Object, Array, Boolean (true/false), أو null.
- مفيش Trailing Comma (فاصلة زيادة بعد آخر عنصر) — ده بيسبب Parsing Error فورًا.
- مفيش تعليقات (Comments) بأي شكل من الأشكال.
- الترميز الافتراضي هو UTF-8.

Objects / Mappings

الـ Object في JSON هو مجموعة أزواج مفتاح-قيمة محاطة بأقواس معقوفة { }، والمفاتيح لازم تكون فريدة منطقيًا (رغم إن بعض الـ Parsers بتقبل تكرار المفتاح وبتاخد آخر قيمة بس).

```
{
  "name": "Ahmed",
  "age": 28,
  "isActive": true
}
```

Arrays / Lists

الـ Array هو قائمة مرتبة من القيم محاطة بأقواس مربعة [ ]، والقيم ممكن تكون من أنواع مختلفة في نفس الـ Array (رغم إن ده مش موصى بيه عمليًا).

```
{
  "tags": ["backend", "python", "api"],
  "scores": [95, 88, 76.5]
}
```

Strings

أي نص لازم يكون محاط بعلامتي تنصيص مزدوجة، ولازم أي حرف خاص جواه يتكتب بطريقة Escaped.

```
{
  "message": "قال: \"مرحبًا\"",
  "path": "C:\\Users\\Ahmed",
  "newline_example": "سطر أول\nسطر ثاني"
}
```

```
}
```

Numbers

الأرقام في JSON مفيش فيها تفرقة صريحة بين Integer وFloat على مستوى الـ Syntax، لكن اللغة اللي بتقرأ الملف هي اللي بتقرر تمثيلها داخليًا.

- لا يوجد دعم لـ Infinity أو NaN — دول مش قيم صحيحة في JSON.
- لا يجوز إضافة صفر زيادة في البداية (زي ٠٠٧) ولا فاصلة بدون رقم قبلها (زي .٥ لازم تتكتب ٠,٥).
- مسموح بالـ Scientific Notation زي ١,٥e10.

```
{ "price": 19.99, "quantity": 3, "discount": 0, "factor": 1.5e3 }
```

Null وBoolean

القيم true وfalse وnull لازم تُكتب بحروف صغيرة (lowercase) بالظبط، أي اختلاف في الحالة (زي True أو NULL) بيعتبر خطأ Syntax.

```
{ "isVerified": true, "deletedAt": null }
```

Nested Objects وNested Lists

JSON بتدعم التداخل (Nesting) اللانهائي نظريًا — Object جوا Array، وArray جوا Object، وهكذا.

```
{
  "user": {
    "name": "Sara",
    "addresses": [
      { "city": "Cairo", "type": "home" },
      { "city": "Giza", "type": "work" }
    ]
  }
}
```

Escaping وUnicode

الحروف الخاصة اللي محتاجة Escape جوا الـ String: علامة التنصيص " ، الـ Backslash \، وحروف التحكم زي \n و\t و\r.
أما الـ Unicode فممكن تتكتب مباشرة (لو الترميز UTF-8) أو عن طريق \uXXXX.

```
{ "emoji_code": "\u2764", "arabic_direct": "مرحبًا" }
```

⚠ خطأ شائع

استخدام علامة تنصيص مفردة \' بدل المزدوجة \" — JSON مش بتقبلها إطلاقًا، على عكس JavaScript العادية.

⚠ خطأ شائع

ترك فاصلة زيادة بعد آخر عنصر في Object أو Array — سبب كلاسيكي لأخطاء الـ Parsing.

حالات حافة (Edge Cases) في الأرقام

فيه تفاصيل دقيقة كتير مبرمجين بيتفاجؤوا بيها لما بيتعاملوا مع أرقام JSON عبر لغات مختلفة:

- الأرقام الصحيحة الكبيرة جدًا (زي معرّفات قواعد بيانات ٦٤-bit): JavaScript بتفقد الدقة لو الرقم أكبر من ٥٣٨٢، فبعض الـ APIs بتحوّل الأرقام دي لـ String عشان تتجنب المشكلة دي، رغم إنها منطقيًا أرقام.
- القيمة -٠: مسموحة في JSON، لكن التعامل معاها بيختلف شوية بين اللغات المختلفة (خصوصًا في المقارنات المنطقية).
- عدد الأرقام العشرية بعد الفاصلة: JSON نفسها مالهاش حد أقصى، لكن دقة التمثيل بتعتمد بالكامل على نوع البيانات اللي اللغة المستقبلة هتستخدمه (float64 عادةً).

> **ملاحظة**
>
> لو شغال بأرقام مالية حساسة (فلوس، عملات)، فكّر تستقبلها كـ String في الـ JSON وتحولها لـ Decimal في الكود بدل ما تعتمد على float العادي، لتفادي مشاكل التقريب الشهيرة.

Flow Style في YAML بالتفصيل

زي ما ذكرنا، YAML بتدعم أسلوب مضغوط (Flow Style) شبيه بـ JSON تمامًا جوا ملف Block Style عادي، وده مفيد جدًا للقوائم أو الكائنات الصغيرة اللي مش محتاجة تفاصيل كتير:

```yaml
# Block Style (الأسلوب المعتاد)
ports:
  - 8000
  - 8001

# Flow Style (مكافئ تمامًا، لكن أكثر إيجازًا)
ports: [8000, 8001]

# ممكن تدمج الاتنين في نفس الملف حسب الحاجة
server:
  host: localhost
  allowed_ips: [127.0.0.1, 192.168.1.1]
```

> **أفضل الممارسات**
>
> استخدم Flow Style للقوائم القصيرة جدًا أو الكائنات البسيطة (زي إحداثيات أو نطاق أرقام)، واستخدم Block Style لأي حاجة فيها تفاصيل أو تعليقات — كده تجمع بين الإيجاز والوضوح حسب الموقف.

ثانيًا: بنية YAML

القواعد الأساسية

- البنية معتمدة بالكامل على المسافات البادئة (Indentation) — استخدام Tab ممنوع منعًا باتًا في أغلب الـ Parsers.
- عدد المسافات مش مهم بالتحديد (٢ أو ٤) لكن لازم يكون ثابت ومتسق في نفس المستوى.
- النقطتين : بعدها لازم مسافة واحدة على الأقل قبل القيمة.
- التعليقات بتبدأ بعلامة #.
- ملف YAML ممكن يبدأ اختياريًا بـ --- (بداية مستند) وينتهي بـ ... (نهاية مستند).

Mappings (المكافئ لـ Objects)

```yaml
name: Ahmed
age: 28
isActive: true
```

Lists (المكافئ لـ Arrays)

في YAML، عناصر القائمة بتتكتب بعلامة الشرطة - مع مسافة بعدها:

```yaml
tags:
  - backend
  - python
  - api
```

وفي فيه أسلوب مضغوط (Flow Style) شبه JSON تمامًا، ممكن تستخدمه جوا YAML عادي:

```yaml
tags: [backend, python, api]
```

Strings

النصوص في YAML ممكن تتكتب من غير علامات تنصيص خالص في أغلب الحالات، لكن فيه ٣ طرق:

- Plain (بدون علامات): name: Ahmed
- Single-quoted: بتحافظ على النص حرفيًا، والطريقة الوحيدة لعمل Escape جواها هي تكرار علامة التنصيص نفسها ''.
- Double-quoted: بتدعم Escape Sequences زي \n و\t بنفس طريقة JSON.

```yaml
plain_text: مرحبًا بالجميع
single: 'هنا سوى ''للتنصيص'' لا يوجد Escape'
double: "سطر أول\nسطر ثاني"
```

Numbers, Boolean, Null

```yaml
price: 19.99
quantity: 3
is_active: true      # أو yes / True / on حسب النسخة
```

```
deleted_at: null      # أو سيبها فاضية أو
```

> ⚠ **خطأ شائع**
> كتابة قيمة زي NO أو YES أو ON أو OFF كنص عادي من غير علامات تنصيص — في YAML 1.1 هتتفسر تلقائيًا كـ Boolean، وده معروف باسم مشكلة Norway Problem. الحل: استخدم علامات تنصيص صريحة زي "NO" لو قصدك نص فعلاً.

Nested Objects وNested Lists

التداخل في YAML بيتحدد بالكامل عن طريق مستوى الإزاحة (Indentation Level):

```
user:
  name: Sara
  addresses:
    - city: Cairo
      type: home
    - city: Giza
      type: work
```

Multi-line Strings

YAML بتدعم كتابة نص متعدد الأسطر بطريقتين رئيسيتين:

- Literal Style (|): بتحافظ على كل الأسطر الجديدة (Newlines) زي ما هي.
- Folded Style (>): بتحول الأسطر الجديدة لمسافات، وبتحافظ بس على الأسطر الفاضية كفواصل فقرات.

```
description_literal: |
  السطر الأول
  السطر الثاني
  سيبقيان منفصلين تمامًا

description_folded: >
  السطر الأول
  السطر الثاني
  هيتحولوا لسطر واحد متصل
```

Documents متعددة في ملف واحد

YAML الوحيدة اللي بتدي إمكانية وضع أكتر من مستند منفصل في نفس الملف عن طريق الفاصل ---، وده مفيد جدًا في Kubernetes مثلًا لتعريف أكتر من Resource في ملف واحد:

```
apiVersion: v1
kind: Service
metadata:
  name: my-service
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-deployment
```

Special Characters وFormatting

- لازم تتجنب استخدام حروف زي : أو # جوا نص Plain من غير علامات تنصيص، لأنها ممكن تتفسر كبداية تعليق أو Mapping جديد.
- الترميز الافتراضي UTF-8، وبيدعم Unicode مباشرة زي JSON.
- لازم الحفاظ على تناسق المسافات بين كل عناصر نفس المستوى، وإلا هيحصل YAMLError واضح بيوصف مكان المشكلة.

> ☑ **أفضل الممارسات**
>
> اختار عدد مسافات ثابت (الأشهر عمليًا: مسافتين) والتزم بيه في كل الملف، ومعظم الـ Editors بتقدر تعرض المسافات (Whitespace) بصريًا عشان تتجنب الأخطاء.

مقارنة سريعة: بنية JSON مقابل بنية YAML

YAML	JSON	العنصر
Indentation فقط	{ }	تحديد الكائن
- أو [ ] (Flow Style)	[ ]	تحديد القائمة
سطر جديد أو فاصلة (Flow Style)	فاصلة ,	الفاصل بين العناصر
اختيارية غالبًا	إلزامية (" ")	علامات التنصيص للنصوص
# مدعومة	غير مدعومة	التعليقات
| أو > كأسلوب مخصص	\n داخل String	نصوص متعددة الأسطر

الأسئلة الشائعة

❓ هل ينفع أستخدم Tab بدل Spaces في YAML؟

لا، أغلب الـ Parsers الرسمية بترفض ملف فيه Tab للـ Indentation وبترمي خطأ فورًا؛ استخدم Spaces دايمًا.

❓ ليه مفيش علامات تنصيص إلزامية في YAML زي JSON؟

لأن الفلسفة الأساسية لـ YAML هي تقليل الرموز الزيادة قدر الإمكان لصالح القراءة البشرية، لكن الثمن هو إن بعض القيم ممكن تتفسر بشكل غير متوقع (زي مشكلة Norway) لو معتمدتش على علامات التنصيص وقت الحاجة.

أخطاء شائعة

⚠ خطأ شائع

خلط المسافات مع الـ Tabs في نفس ملف YAML، وده بيسبب خطأ صعب التتبع بصريًا.

⚠ خطأ شائع

نسيان المسافة بعد النقطتين : في key:value — YAML من غير مسافة بيتفسر بشكل خاطئ.

أفضل الممارسات

☑ أفضل الممارسات

في JSON، استخدم أداة Formatter تلقائية (زي Prettier) لضمان اتساق التنسيق قبل أي Commit.

☑ أفضل الممارسات

في YAML، فعّل خاصية عرض المسافات في الـ Editor بتاعك، واستخدم Linter زي yamllint للتأكد من صحة الملف قبل الاستخدام.

ملخص الفصل

- JSON تعتمد على رموز صريحة ({}, [], ":", ",") ومفيش فيها تعليقات ولا Trailing Commas.
- YAML تعتمد على المسافات البادئة بدل الرموز، وبتدعم التعليقات والنصوص متعددة الأسطر والمستندات المتعددة في ملف واحد.
- أخطر خطأ شائع في YAML هو الخلط بين Tabs وSpaces، وأخطر خطأ شائع في JSON هو الفاصلة الزيادة بعد آخر عنصر.

تمارين عملية

تمرين ١: حوّل الكائن التالي من JSON إلى YAML يدويًا: { "city": "Cairo", "population": 10000000, "isCapital": true }

الحل المقترح

city: Cairo population: 10000000 isCapital: true

تمرين ٢: ليه القيمة on: true في YAML ممكن تسبب مشكلة لو المقصود منها إنها اسم مفتاح (Key) عادي زي حالة تشغيل جهاز؟

الحل المقترح

لأن on قد تُفسَّر تلقائيًا كقيمة Boolean بدل نص، خصوصًا في YAML 1.1؛ الحل هو وضعها بين علامتي تنصيص "on" لو المقصود نص حرفي.

الفصل الثالث — Core Concepts — المفاهيم الأساسية

قبل ما نروح للكود في الفصل الجاي، لازم نفهم المفاهيم النظرية اللي بتحكم عمل أي صيغة تسلسل بيانات (Serialization Format)، لأن الفهم ده هو اللي هيفرق بينك وبين مطور بيحفظ الأوامر عن ظهر قلب من غير ما يفهم بيحصل إيه فعليًا تحت الغطا.

Serialization وDeserialization

Serialization هي عملية تحويل هيكل بيانات موجود في ذاكرة البرنامج (زي Object في Python أو Dictionary) لصيغة نصية أو ثنائية ممكن تتخزن أو تتبعت عبر الشبكة — ده بالظبط اللي بيحصل لما تستخدم json.dumps() في Python.

Deserialization هي العملية العكسية: تاخد النص المخزّن أو المستلم وتحوّله تاني لهيكل بيانات في الذاكرة تقدر تتعامل معاه في الكود — ده اللي بيحصل مع json.loads().

> **ملاحظة**
>
> المصطلح Marshalling وUnmarshalling بيُستخدموا أحيانًا كمرادفات لـ Serialization وDeserialization خصوصًا في لغات زي Go وC#.

Parsing

الـ Parsing هي عملية تحليل النص حرف بحرف (أو Token by Token) للتأكد إنه متوافق مع قواعد الصيغة (Grammar)، وبناء هيكل بيانات مؤقت (زي Abstract Syntax Tree) قبل ما يتحول للكائن النهائي في الذاكرة. أي Parser بيمر عادةً بمرحلتين:

- Lexical Analysis (Tokenizing): تقسيم النص لوحدات صغيرة (Tokens) زي الأقواس، الفواصل، النصوص، الأرقام.
- Syntactic Analysis: التأكد إن ترتيب الـ Tokens دي متوافق مع قواعد الصيغة، وبناء الهيكل النهائي.

Encoding وDecoding

الترميز (Encoding) هو تحويل الأحرف لبايتات قابلة للتخزين أو النقل (زي UTF-8)، والفك (Decoding) هو العملية العكسية. الموضوع ده مهم جدًا في JSON وYAML لأن المواصفة الرسمية لكل منهم بتفترض إن الترميز الافتراضي هو UTF-8، وأي اختلاف (زي UTF-16 بدون تحديد صريح) ممكن يسبب مشاكل عند القراءة بين أنظمة مختلفة.

Hierarchical Data وStructured Data وNested Data

الفكرة الأساسية اللي JSON وYAML بيمثلوها هي البيانات الهرمية (Hierarchical Data) — يعني بيانات فيها علاقة أب-ابن (Parent-Child) زي شجرة (Tree)، على عكس البيانات الجدولية (Tabular Data) زي CSV اللي بتكون مسطحة (Flat) في صفوف وأعمدة.

- Structured Data: بيانات ليها شكل وقواعد واضحة (مفاتيح معروفة، أنواع محددة).
- Nested Data: بيانات جوا بيانات — Object جوا Object، أو Array جوا Object، إلخ، وده اللي بيدّي الصيغتين القدرة على تمثيل أي هيكل معقد زي شجرة عائلة أو ملف إعداد كامل لتطبيق ضخم.

مفاهيم خاصة بـ YAML: Anchors وAliases وTags وMerge Keys

دي المفاهيم اللي بتدي YAML قوة أكبر من JSON في تقليل التكرار، ومحتاجين نفهمها كويس جدًا لأنها بتظهر كتير في ملفات Kubernetes وDocker Compose وAnsible المعقدة.

Anchors (&) وAliases (*)

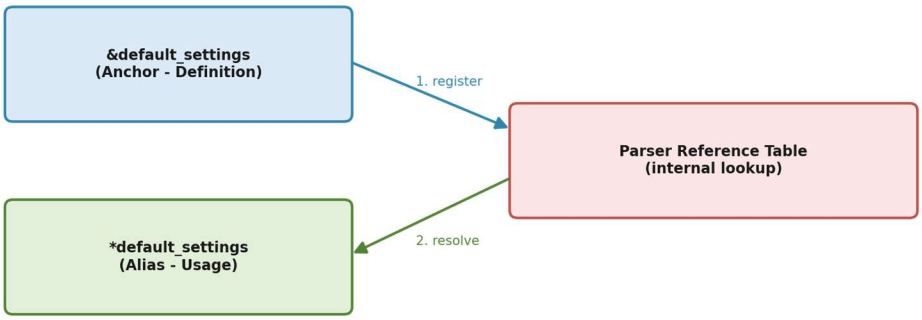
الـ Anchor بيسمحلك تعرّف قيمة أو هيكل مرة واحدة وتديله اسم، وبعدين تستخدم الاسم ده (Alias) في أي مكان تاني بالملف للإشارة لنفس القيمة بالظبط:

```yaml
defaults: &default_settings
  adapter: postgres
  timeout: 30

development:
  <<: *default_settings
  database: dev_db

production:
  <<: *default_settings
  database: prod_db
```

في المثال ده، development وproduction كلاهما بياخد نفس القيم الافتراضية من default_settings من غير ما نكررها، وده تطبيق مباشر لمبدأ DRY (Don't Repeat Yourself) جوا ملف الإعداد نفسه.



شكل ٢٫٣: الـ Parser بيسجل الـ Anchor في جدول مرجعي، وبيرجعله وقت ما يقابل Alias مطابق

Merge Keys (<<)

علامة << هي اللي بتعمل "دمج" (Merge) لـ Mapping المشار إليه بالـ Alias جوا Mapping تاني، زي ما شفنا فوق. مهم نعرف إن << مش جزء رسمي من مواصفة YAML الأساسية (Core Schema)، لكنها اتفاق واسع الانتشار بين أغلب الـ Parsers ومدعومة في PyYAML وruamel.yaml.

> ⚠ **خطأ شائع**
>
> الاعتماد على Merge Keys في مشروع من غير التأكد إن الـ Parser المستخدم بيدعمها فعليًا — بعض الأدوات الصارمة اللي بتلتزم بالمواصفة الأساسية بس ممكن ترفضها.

Tags

الـ Tag في YAML بتحدد نوع البيانات بشكل صريح لو الـ Parser مش هيقدر يستنتجه تلقائيًا. الصيغة بتبدأ بعلامة !!:

```
explicit_string: !!str 42
explicit_int: !!int "42"
custom_object: !Person
  name: Ahmed
  age: 30
```

الـ Tags المخصصة زي !Person بتُستخدم مع Custom Constructors في لغات البرمجة (هنشرحها بالتفصيل في فصل Python) عشان تحوّل جزء من YAML مباشرة لكائن مخصص في الكود بدل Dictionary عادي.

Human Readability مقابل Machine Readability

كل صيغة بتاخد قرار موازنة (Trade-off) بين الاتنين:

- JSON بتميل ناحية Machine Readability: الرموز الصريحة بتخلي الـ Parsing أسرع وأبسط للآلة، حتى لو كانت أقل راحة بصريًا للإنسان في الملفات الكبيرة.
- YAML بتميل ناحية Human Readability: الاعتماد على المسافات والتعليقات بيريّح عين المبرمج، لكنه بيحمّل الـ Parser تعقيد أكبر (لازم يتتبع المستويات بدقة).

> **ملاحظة**
>
> مفيش صيغة "أفضل" في المطلق؛ الاختيار الصح دايمًا مبني على: مين اللي هيتعامل مع الملف ده أكتر، إنسان بيعدّل يدويًا، ولا آلة بتقرأ وتكتب آلاف المرات في الثانية؟

كيف تعمل الـ Parsers داخليًا (High-Level)

بشكل مبسط، أي JSON أو YAML Parser بيمر بالخطوات دي:

١. القراءة: تحميل الملف كسلسلة بايتات، وفك الترميز (Decoding) لنص (Unicode String).

٢. Tokenizing: تقسيم النص لوحدات صغيرة زي الأقواس، النقطتين، الفواصل، النصوص، الأرقام.

٣. Parsing/Building Tree: بناء هيكل شجري مؤقت يمثّل التداخل (Objects جوا Arrays جوا Objects...).

٤. Construction: تحويل الشجرة المؤقتة لهيكل بيانات نهائي في اللغة المستخدمة (Dictionary/List في Python مثلاً).



شكل ١٫٣: مراحل عمل أي Parser من البايتات الخام إلى الكائن النهائي في الذاكرة

في حالة YAML، فيه خطوة إضافية معقدة أكتر بسبب الـ Anchors والـ Aliases: الـ Parser لازم يحتفظ بـ "جدول مرجعي" (Reference Table) للـ Anchors أثناء القراءة، عشان لما يوصل لـ Alias يقدر يرجع للقيمة الأصلية ويستخدمها من غير ما يعيد تحليلها من الصفر.

الأسئلة الشائعة

❓ هل Anchors وAliases موجودين في JSON؟

لا، غير موجودين إطلاقًا في JSON. لو محتاج نفس الفكرة في JSON، الحل الشائع هو استخدام أدوات خارجية (زي JSON References $ref في JSON Schema) لكنها مش جزء من مواصفة JSON نفسها.

❓ هل الـ Merge Key << بتشتغل مع كل الـ YAML Parsers؟

لا، هي اتفاق منتشر جدًا (خصوصًا في PyYAML وruamel.yaml) لكنها مش جزء رسمي من كل تطبيقات المواصفة، فلو بتشتغل على مشروع بيستخدم Parser صارم يلتزم بالمواصفة الأساسية بس، تأكد إنه بيدعمها الأول.

أخطاء شائعة

⚠ خطأ شائع

استخدام Alias (*) قبل ما يتعرّف الـ Anchor (&) المرتبط بيه في الملف — الترتيب مهم، الـ Anchor لازم يظهر أولًا.

⚠ خطأ شائع

الخلط بين Deserialization وValidation — تحويل النص لكائن في الذاكرة بنجاح مايعنيش إن البيانات دي "صحيحة منطقيًا"؛ لازم خطوة Validation منفصلة (هنشرحها في الفصل الخامس).

أفضل الممارسات

أفضل الممارسات

استخدم Anchors وAliases في ملفات YAML الكبيرة (زي docker-compose.yml مع عدة خدمات متشابهة) لتقليل التكرار وتسهيل الصيانة.

أفضل الممارسات

لا تفرط في استخدام Tags المخصصة إلا لو محتاج فعلًا تحويل تلقائي لكائنات مخصصة؛ الإفراط فيها بيقلل من قابلية نقل الملف بين أدوات مختلفة.

ملخص الفصل

- Serialization/Deserialization هي عملية التحويل بين هيكل البيانات في الذاكرة والنص المخزّن، وParsing هي العملية التقنية اللي بتحقق التحويل ده.
- YAML بتضيف مفاهيم متقدمة (Anchors, Aliases, Merge Keys, Tags) عشان تقلل التكرار وتدعم تحويل مخصص، مش موجودة في JSON.
- كل صيغة بتاخد قرار موازنة بين القراءة البشرية والقراءة الآلية، وده اللي بيحدد أفضل استخدام لكل منهم.

تمارين عملية

تمرين ١: اكتب مثال YAML فيه Anchor لخدمة قاعدة بيانات، واستخدمه في خدمتين مختلفتين (dev وtest) بدون تكرار الإعدادات.

الحل المقترح

```
:db_defaults: &db  engine: postgres  port: 5432 dev:  <<: *db  name: dev_db test:  <<: *db  name
test_db
```

تمرين ٢: ليه بنقول إن نجاح عملية Deserialization لا يضمن صحة البيانات منطقيًا؟

الحل المقترح

لأن الـ Parser بيتحقق فقط من صحة الـ Syntax (الشكل الشكلي للنص)، لكنه لا يتحقق من القواعد المنطقية للتطبيق (مثل أن الحقل age لازم يكون رقم موجب) — وده دور طبقة الـ Validation المنفصلة.

الفصل الرابع — التعامل مع Python

الفصل ده عملي بالكامل: هنشرح إزاي تتعامل مع JSON وYAML جوا Python بالتفصيل، مع كل Parameter مهم، وBest Practices، وأخطاء شائعة.

أولًا: JSON في Python — وحدة json

وحدة json موجودة في المكتبة القياسية (Standard Library) من غير الحاجة لتثبيت أي حاجة، وبتوفر ٤ دوال أساسية: load وloads للقراءة، وdump وdumps للكتابة.

json.loads() و json.load()

loads() (بحرف s في الآخر يعني String) بتاخد نص JSON وترجع كائن Python. أما load() فبتاخد File Object مباشرة وتقرأ منه.

```
import json

# من نص
data = json.loads('{"name": "Ahmed", "age": 30}')
print(data["name"])  # Ahmed

# من ملف
with open('data.json', 'r', encoding='utf-8') as f:
    data = json.load(f)
```

json.dumps() و json.dump()

dumps() بتحول كائن Python لنص JSON، وdump() بتكتبه مباشرة في ملف.

```
import json

data = {"name": "Ahmed", "age": 30, "city": "القاهرة"}

json_string = json.dumps(data, indent=4, ensure_ascii=False)
print(json_string)

with open('output.json', 'w', encoding='utf-8') as f:
    json.dump(data, f, indent=4, ensure_ascii=False)
```

أهم الـ Parameters

- indent: عدد المسافات للـ Formatting (لو اتسابت None، الناتج بيبقى في سطر واحد بدون تنسيق).
- ensure_ascii: افتراضيًا True، وده معناه إن أي حرف غير ASCII (زي العربي) هيتحول لـ \uXXXX. لازم تخليها False عشان تشوف النص العربي بشكله الطبيعي.
- sort_keys: لو True، هيرتب المفاتيح أبجديًا في الناتج.
- separators: تحديد الفاصل بين العناصر وبين المفتاح والقيمة، مفيد لتقليل حجم الملف (Compact JSON) عن طريق (',', ':') بدل الافتراضي (', ', ': ').

- object_hook: دالة بتستقبل كل Dictionary اتقرا وتقدر تحوّله لكائن مخصص (Custom Object) أثناء الـ Deserialization.
- parse_float / parse_int / parse_constant: بتسمحلك تغيّر إزاي الأرقام أو القيم الخاصة (NaN, Infinity) بتتفسر، مثلًا تحويل كل الأرقام العشرية لـ Decimal بدل float عشان دقة أعلى ماليًا.

```
import json
from decimal import Decimal

# استخدام parse_float للحفاظ على الدقة في العمليات المالية
data = json.loads('{"price": 19.99}', parse_float=Decimal)
print(type(data['price']))  # <class 'decimal.Decimal'>
```

> ⚠ **خطأ شائع**
> نسيان ensure_ascii=False لما تشتغل بنصوص عربية — هيطلعلك الملف مليان \u0645\u0631... بدل النص العربي الطبيعي.

Custom Encoder و Custom Decoder

لما تحتاج تحول كائن مش مدعوم افتراضيًا (زي datetime أو كائن مخصص) لازم تعمل Encoder مخصص عن طريق وراثة JSONEncoder:

```
import json
from datetime import datetime

class CustomEncoder(json.JSONEncoder):
    def default(self, obj):
        if isinstance(obj, datetime):
            return obj.isoformat()
        return super().default(obj)

data = {"created_at": datetime.now()}
print(json.dumps(data, cls=CustomEncoder))
```

وعلى العكس، لعمل Decoder مخصص بتستخدم object_hook مباشرة، أو تورّث JSONDecoder لتحكم أعمق في مراحل الـ Parsing نفسها.

ثانيًا: YAML في Python

على عكس JSON، YAML مش جزء من المكتبة القياسية في Python، فلازم تثبّت مكتبة خارجية. أشهر اتنين: PyYAML وruamel.yaml.

PyYAML

المكتبة الأكثر شيوعًا وانتشارًا، بسيطة الاستخدام، لكنها بتفقد التنسيق الأصلي (Comments, Order) لما تعيد كتابة الملف.

```
import yaml

with open('config.yaml', 'r', encoding='utf-8') as f:
    data = yaml.safe_load(f)

print(data['database']['host'])

# الكتابة
with open('output.yaml', 'w', encoding='utf-8') as f:
    yaml.safe_dump(data, f, allow_unicode=True, sort_keys=False)
```

SafeLoader مقابل FullLoader مقابل UnsafeLoader

- SafeLoader (أو yaml.safe_load): بيحمّل بس أنواع بيانات بسيطة وآمنة (Dict, List, String, Number, Boolean, None). هو الخيار الافتراضي الموصى بيه دايمًا لأي بيانات مش موثوق فيها ١٠٠٪.
- FullLoader: بيدعم بنى Python أوسع، لكنه لسه بيمنع تنفيذ كود عشوائي (منذ PyYAML 5.1 بقى الافتراضي بدل Loader القديم غير الآمن).
- UnsafeLoader (أو yaml.load بدون تحديد Loader في نسخ قديمة): بيقدر يبني أي كائن Python عشوائي من الملف، وده بيفتح ثغرة أمنية خطيرة لو الملف جاي من مصدر غير موثوق — هنشرح التفاصيل الكاملة في فصل الأمان.

> ⚠ **خطأ شائع**
>
> استخدام yaml.load(file) من غير تحديد Loader صريح في نسخ PyYAML القديمة (قبل ٥,١) — ده كان بيستخدم UnsafeLoader افتراضيًا، وهي واحدة من أشهر الثغرات الأمنية في تاريخ مكتبات Python.

> ☑ **أفضل الممارسات**
>
> استخدم yaml.safe_load() دايمًا إلا لو عندك سبب قوي جدًا ومتحكم بالكامل في مصدر الملف.

ruamel.yaml

مكتبة أحدث ومصممة خصيصًا لحل مشكلة PyYAML الأساسية: الحفاظ على التنسيق الأصلي (Comments, ترتيب المفاتيح, الأسلوب الأصلي Block/Flow) لما تقرأ الملف وتعدّل فيه وتعيد كتابته. دي مهمة جدًا في أدوات زي Ansible وKubernetes tooling اللي بتعدّل ملفات YAML برمجيًا وعايزة تحافظ على تعليقات المطور الأصلية.

```
from ruamel.yaml import YAML

yaml = YAML()
yaml.preserve_quotes = True
```

```
with open('config.yaml') as f:
    data = yaml.load(f)

data['database']['port'] = 5433  # تعديل قيمة واحدة بس

with open('config.yaml', 'w') as f:
    yaml.dump(data, f)  # التعليقات والترتيب هيفضلوا زي ما هما
```

Custom Constructors و Custom Representers

زي ما JSON عندها YAML، Custom Encoder/Decoder عندها نفس الفكرة بمسميات مختلفة:

- Constructor: دالة بتحدد إزاي Tag معين (زي !Person) يتحول لكائن Python فعلي وقت القراءة.
- Representer: دالة بتحدد إزاي كائن Python مخصص يتحول لتمثيل YAML وقت الكتابة.

```
import yaml

class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

def person_constructor(loader, node):
    values = loader.construct_mapping(node)
    return Person(values['name'], values['age'])

yaml.SafeLoader.add_constructor('!Person', person_constructor)

data = yaml.safe_load('name: !Person {name: Ahmed, age: 30}')
```

مقارنة: PyYAML مقابل ruamel.yaml

ruamel.yaml	PyYAML	المعيار
✅ نعم	❌ لا	الحفاظ على التعليقات عند إعادة الكتابة
✅ كامل	بشكل جزئي	الحفاظ على ترتيب المفاتيح الأصلي
آمن بشكل افتراضي	آمن بـ safe_load	الأمان الافتراضي
أقل شيوعًا لكن أدق	الأوسع انتشارًا	الانتشار والاستخدام
أبطأ قليلًا	أسرع نسبيًا	الأداء (Parsing Speed)
تعديل ملفات موجودة مع الحفاظ على شكلها	قراءة/كتابة بسيطة	الاستخدام المثالي

الأسئلة الشائعة

❓ ليه مفيش yaml module جاهزة في Python زي json؟

لأن YAML أعقد بكتير من JSON في مواصفتها، والمجتمع فضّل يسيب الموضوع لمكتبات خارجية متخصصة (PyYAML وruamel.yaml) بدل ما يضيف تعقيد للمكتبة القياسية.

❓ هل ينفع أستخدم json module عادي عشان أقرأ ملف YAML بسيط؟

لا، json module بتتوقع Syntax صارم بالأقواس، ومش هتقدر تفهم Indentation-based YAML. لازم مكتبة YAML مخصصة حتى لو الملف بسيط.

أخطاء شائعة

⚠ خطأ شائع

فتح ملفات بدون تحديد 'encoding='utf-8 صراحة، وده بيسبب مشاكل غريبة على أنظمة تشغيل مختلفة (خصوصًا Windows) مع النصوص العربية.

⚠ خطأ شائع

استخدام yaml.dump() العادي على بيانات فيها نص عربي من غير allow_unicode=True — هيتحول لـ Escape Codes غير مقروءة.

أفضل الممارسات

✅ أفضل الممارسات

استخدم 'encoding='utf-8 صراحة في كل عمليات فتح الملفات، وensure_ascii=False في JSON، وallow_unicode=True في YAML، لضمان ظهور النصوص العربية بشكل صحيح.

> ☑ **أفضل الممارسات**
>
> لو بتبني أداة بتعدّل ملفات YAML موجودة فعلًا (زي Config Editor)، استخدم ruamel.yaml مش PyYAML، عشان محافظتها على التنسيق الأصلي وتعليقات المطورين.

ملخص الفصل

- json module جزء من المكتبة القياسية في Python، وبتقدم load/loads للقراءة وdump/dumps للكتابة، مع دعم Custom Encoder/Decoder للكائنات المعقدة.
- YAML محتاجة مكتبة خارجية: PyYAML للاستخدام العام السريع، وruamel.yaml لما تحتاج تحافظ على شكل الملف الأصلي عند التعديل.
- استخدام safe_load دايمًا بدل load أساسي جدًا للأمان، وهنتعمق فيه أكتر في فصل الأمان.

تمارين عملية

تمرين ١: اكتب كود Python يقرأ ملف JSON فيه تاريخ كـ String بصيغة ISO، ويحوله تلقائيًا لكائن datetime عن طريق object_hook.

> 💡 **الحل المقترح**
>
> = import json from datetime import datetime def hook(d): if 'created_at' in d: d['created_at'] datetime.fromisoformat(d['created_at']) return d data = json.loads(json_text, object_hook=hook)

تمرين ٢: ليه لازم نستخدم safe_load بدل load في PyYAML حتى لو كنا متأكدين إن الملف من داخل الشركة؟

> 💡 **الحل المقترح**
>
> لأن 'من داخل الشركة' مش ضمان كافٍ؛ أي ملف ممكن يتعدل بالخطأ أو يتسرب منه محتوى ضار عبر نظام آخر متصل، والعادة الآمنة (safe_load دايمًا) بتحمي من المفاجآت بغض النظر عن مصدر الملف.

الفصل الخامس — Validation & Schema — التحقق من صحة البيانات

زي ما قلنا في الفصل التالت، نجاح الـ Parsing مايعنيش إن البيانات صحيحة منطقيًا. الفصل ده بيشرح إزاي تضمن إن أي ملف JSON أو YAML بيلتزم بشكل وقواعد محددة قبل ما تستخدمه في تطبيقك.

أولًا: JSON Schema

JSON Schema هي مواصفة رسمية مستقلة (بيها JSON نفسها) بتوصف "شكل" ملف JSON المتوقع: إيه المفاتيح المطلوبة، إيه نوع كل قيمة، إيه القيود عليها. هي بمثابة "عقد" (Contract) بين مُنتج البيانات ومُستهلكها.

Properties وRequired

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "properties": {
    "name": { "type": "string" },
    "age": { "type": "integer", "minimum": 0 }
  },
  "required": ["name", "age"]
}
```

properties بتحدد شكل كل مفتاح متوقع، وrequired بتحدد أي المفاتيح دي لازم تكون موجودة إجباريًا؛ أي مفتاح مش موجود في properties عادةً مسموح بيه إلا لو حددت additionalProperties: false.

Pattern وEnum

```
{
  "email": {
    "type": "string",
    "pattern": "^[^@]+@[^@]+\\.[^@]+$"
  },
  "status": {
    "type": "string",
    "enum": ["active", "inactive", "pending"]
  }
}
```

- pattern: بيتحقق من النص عن طريق Regular Expression.
- enum: بيحصر القيمة المسموحة في مجموعة محددة بس.

OneOf, AnyOf, AllOf

- oneOf: البيانات لازم تطابق واحد بالظبط من الخيارات المذكورة، مش أكتر ومش أقل.
- anyOf: البيانات لازم تطابق خيار واحد على الأقل من الخيارات (ممكن أكتر من واحد).
- allOf: البيانات لازم تطابق كل الخيارات المذكورة في نفس الوقت (بيُستخدم غالبًا لدمج أكتر من Schema في واحد).

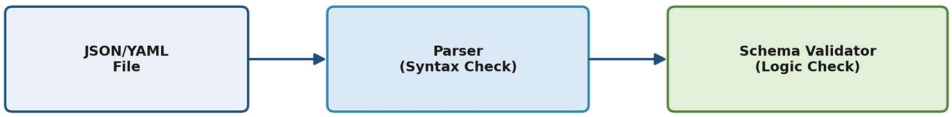
```
{
```

```
  "payment_method": {
    "oneOf": [
      { "type": "object", "properties": { "type": { "const": "credit_card" } } },
      { "type": "object", "properties": { "type": { "const": "paypal" } } }
    ]
  }
}
```

Nested Validation وCustom Validation

JSON Schema بتدعم التحقق المتداخل بشكل كامل — يعني تقدر تحدد Schema لكائن جوا كائن جوا Array، بنفس المرونة اللي JSON نفسها بتدعمها. أما الـ Custom Validation (قواعد منطقية أعقد من مجرد النوع أو الـ Pattern، زي "تاريخ النهاية لازم يكون بعد تاريخ البداية") فمش بتتحل بـ JSON Schema وحدها عادةً، وبتحتاج كود Validation إضافي في التطبيق نفسه بعد التحقق الأساسي.

> **ملاحظة**
>
> أشهر مكتبات تطبيق JSON Schema في Python: jsonschema. في JavaScript/Node.js: ajv وهي من أسرع المكتبات المتاحة.



شكل ١٫٥: التحقق (Validation) خطوة منفصلة تمامًا تحصل بعد نجاح الـ Parsing

ثانيًا: التحقق من صحة YAML

مفيش مواصفة رسمية موازية لـ "YAML Schema" منتشرة بنفس قوة JSON Schema، لكن فيه طرق عملية شائعة جدًا للتحقق:

طرق التحقق الشائعة

- تحويل YAML لـ JSON أولًا (لأن YAML Superset لـ JSON) ثم تطبيق JSON Schema عليها — الطريقة الأكثر انتشارًا عمليًا.
- استخدام Pydantic في Python: تحمّل الـ YAML كـ Dictionary عادي، ثم تمرره على Model معرّف بـ Pydantic اللي بيتحقق من الأنواع والقيود تلقائيًا.
- استخدام yamllint: أداة بتتحقق من صحة الـ Syntax نفسه (Indentation, تنسيق) مش من المنطق أو الشكل.
- Schemas مخصصة لكل أداة: Kubernetes مثلًا عندها Schema داخلية خاصة بيها (OpenAPI-based) للتحقق من صحة أي Manifest قبل تطبيقه بالـ kubectl apply --dry-run.

```
from pydantic import BaseModel
import yaml

class DatabaseConfig(BaseModel):
    host: str
    port: int
    name: str

with open('config.yaml') as f:
    raw = yaml.safe_load(f)

config = DatabaseConfig(**raw["database"])  # هيرمي خطأ واضح لو فيه مشكلة في الأنواع أو مفتاح ناقص
```

أفضل الممارسات

استخدم Pydantic (أو dataclasses مع مكتبة تحقق) في أي مشروع Python جاد بدل الاعتماد على Dictionary خام بدون تحقق — كده بتكتشف الأخطاء في ملف الإعداد وقت بدء التشغيل مش وسط التنفيذ.

Common Practices

- دمج yamllint في الـ CI Pipeline عشان أي Pull Request فيه ملف YAML بمشكلة تنسيق يترفض تلقائيًا.
- توثيق الـ Schema المتوقعة في تعليقات أعلى الملف نفسه، حتى لو مفيش تحقق آلي صارم.
- استخدام أدوات الـ IDE (زي VS Code مع Extension الخاص بـ Kubernetes أو YAML) اللي بتوفر Schema Validation مباشرة أثناء الكتابة.

الأسئلة الشائعة

❓ هل ينفع أستخدم JSON Schema مباشرة على ملف YAML؟

نعم، بشرط تحوّل الـ YAML لهيكل بيانات (Dictionary/List) أولًا عن طريق أي YAML Parser، وبعدين تمرر الهيكل ده على مكتبة JSON Schema — لأن أدوات التحقق دي بتشتغل على هيكل البيانات في الذاكرة مش على النص الخام.

❓ إيه الفرق بين Validation وParsing؟

الـ Parsing بيتأكد إن النص "شكله" صحيح حسب قواعد اللغة (Syntax) ويحوّله لهيكل بيانات، أما الـ Validation فبيتأكد إن هيكل البيانات ده "محتواه" صحيح منطقيًا حسب قواعد التطبيق (مثلاً: الحقل age لازم يكون رقم موجب أقل من ١٥٠).

أخطاء شائعة

⚠ خطأ شائع

الاعتماد على نجاح json.loads() أو yaml.safe_load() كدليل على صحة البيانات، من غير أي خطوة Validation إضافية بعدها.

⚠ خطأ شائع

كتابة Schema معقدة جدًا من البداية بدل البدء بأبسط شكل ممكن وتوسيعه تدريجيًا حسب الحاجة الفعلية.

أفضل الممارسات

☑ أفضل الممارسات

اجعل الـ Validation جزء أساسي من مرحلة بدء التشغيل (Startup) في أي تطبيق، مش خطوة اختيارية لاحقة.

☑ أفضل الممارسات

استخدم رسائل خطأ واضحة عند فشل الـ Validation، توضح بالظبط أي مفتاح ناقص أو أي قيمة غير صالحة، بدل رسالة عامة غامضة.

ملخص الفصل

- JSON Schema هي المعيار الرسمي والأقوى للتحقق من شكل ملفات JSON، وبتدعم properties وrequired وpattern وenum وoneOf/anyOf/allOf.
- YAML لا تملك مواصفة تحقق رسمية موازية بنفس القوة، لكن الممارسة الشائعة هي تحويلها لهيكل بيانات ثم تطبيق JSON Schema أو Pydantic عليها.
- التحقق (Validation) مرحلة منفصلة تمامًا عن التحليل (Parsing)، ولازم تُنفَّذ صراحة، مش افتراض إنها بتحصل تلقائيًا.

تمارين عملية

تمرين ١: اكتب JSON Schema بسيط للتحقق من كائن مستخدم فيه name (نص إجباري) وage (رقم صحيح من ٠ لـ ١٢٠) وemail (نص اختياري).

الحل المقترح

```
{
  "type": "object",
  "properties": {
    "name": {"type": "string"},
    "age": {
      "type": "integer",
      "minimum": 0,
      "maximum": 120
      },
    "email": {"type": "string"}
    },
  "required": ["name", "age"]
}
```

تمرين ٢: اكتب Pydantic Model بسيط يعادل الـ Schema اللي كتبته فوق.

الحل المقترح

```
from pydantic import BaseModel
from typing import Optional
class User(BaseModel):
  name: str
  age: int
  mail: Optional[str] = None
```

الفصل السادس — الاستخدامات العملية

الفصل ده هنشوف فيه إزاي JSON وYAML بيُستخدموا فعليًا في كل جزء تقريبًا من الـ Software Stack الحديث، من الـ Backend لحد الـ Cloud.

REST APIs وGraphQL

JSON هي الصيغة الافتراضية لأي REST API تقريبًا؛ الطلب (Request) والاستجابة (Response) بيتبادلوا كـ JSON مع Header اسمه Content-Type: application/json. أما GraphQL فبيستخدم JSON كصيغة نقل (Transport) للاستعلامات والاستجابات، رغم إن الـ Query نفسها بصيغة خاصة بيها.

```
POST /api/users HTTP/1.1
Content-Type: application/json

{ "name": "Ahmed", "email": "ahmed@example.com" }
```

Backend وFrontend Development

في الـ Backend (FastAPI, Django, Express)، JSON هي الصيغة الافتراضية لاستقبال وإرجاع البيانات. في الـ Frontend، JavaScript بتتعامل مع JSON بشكل طبيعي جدًا عن طريق JSON.parse() وJSON.stringify()، وده سبب أساسي في انتشارها الواسع.

Python Applications وMachine Learning وData Science

- ملفات إعداد النماذج (Model Config): معظم أدوات الـ ML (Hydra, MLflow, DVC) بتستخدم YAML لتعريف الـ Hyperparameters وإعدادات التجارب.
  - تخزين النتائج والـ Metrics: JSON مناسبة أكتر لتخزين نتائج التجارب لأنها هتتقرأ آليًا في أغلب الأحيان (Dashboards, Logging Systems).
  - Notebooks: ملفات Jupyter Notebook (.ipynb) نفسها مكتوبة بصيغة JSON بالكامل تحت الغطا.

MLOps وDocker وDocker Compose

Docker Compose بيستخدم YAML لتعريف الخدمات المتعددة (Services) والشبكات (Networks) والتخزين (Volumes) في تطبيق واحد:

```yaml
version: "3.9"
services:
  api:
    build: ./api
    ports:
      - "8000:8000"
    environment:
      - DATABASE_URL=postgres://db:5432/mydb
    depends_on:
      - db
  db:
    image: postgres:16
    volumes:
      - db_data:/var/lib/postgresql/data
volumes:
  db_data:
```

Kubernetes

كل الـ Manifests في Kubernetes (Deployments, Services, ConfigMaps, Secrets) بتتكتب بـ YAML. مثال Deployment مبسط:

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ml-api
spec:
  replicas: 3
  selector:
    matchLabels:
      app: ml-api
  template:
    metadata:
      labels:
        app: ml-api
    spec:
      containers:
        - name: ml-api
          image: myregistry/ml-api:v2
          ports:
            - containerPort: 8000
```

CI/CD وGitHub Actions

كل Pipeline في GitHub Actions معرّف بملف YAML جوا .github/workflows/:

```yaml
name: Run Tests
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install -r requirements.txt
      - run: pytest
```

Environment Configuration و Cloud Platforms

- AWS CloudFormation: بيدعم YAML وJSON على حد سواء لتعريف البنية التحتية (Infrastructure as Code).
- Google Cloud Deployment Manager: بيستخدم YAML بشكل أساسي.
- Terraform: بيستخدم HCL بشكل أساسي، لكن بيدعم استيراد ملفات JSON كبديل.
- متغيرات البيئة (Environment Variables): مش JSON ولا YAML أساسًا، لكن أدوات كتير (Docker Compose, dotenv) بتقرأ إعدادات البيئة من ملفات YAML أو JSON مركزية.

Logging و VS Code Settings و IDE Configuration

- إعدادات Logging في Python (logging.config.dictConfig) غالبًا بتتحمل من ملف YAML أو JSON منفصل عن الكود.
- settings.json في VS Code هي ملف JSON بيتحكم في كل إعدادات المحرر.
- ملفات .eslintrc أو .prettierrc ممكن تتكتب بصيغة JSON أو YAML حسب اختيار المطور.

Microservices و Data Exchange

في بنية الـ Microservices، JSON هي وسيلة التواصل الأساسية بين الخدمات المختلفة عبر HTTP أو Message Queues (RabbitMQ, Kafka)، بينما YAML بتُستخدم لتعريف كل خدمة وإعداداتها والبنية التحتية اللي بتشغّلها (Helm Charts, Kubernetes Manifests).

الأسئلة الشائعة

؟ ليه GitHub Actions اختارت YAML مش JSON؟

لأن ملفات الـ CI/CD بيعدّلها المطورين يدويًا بشكل متكرر، ومحتاجة تعليقات لتوضيح كل خطوة، وده بالظبط المجال اللي YAML بتتفوق فيه.

؟ هل ينفع أستبدل YAML بـ JSON في Docker Compose أو Kubernetes؟

نعم تقنيًا لأن YAML Superset لـ JSON، لكن عمليًا مفيش حد بيعمل كده لأنك هتفقد ميزة التعليقات وسهولة القراءة اللي أصلًا هي سبب اختيار YAML في المجال ده.

أخطاء شائعة

⚠ خطأ شائع

استخدام JSON في ملفات إعداد كبيرة معقدة (زي Kubernetes) بدون تعليقات، وده بيصعّب على الفريق فهم القرارات وراء كل إعداد بعد فترة.

⚠ خطأ شائع

نسخ إعدادات بيئة الإنتاج (Production) وتعديلها يدويًا لبيئة التطوير (Development) بدل استخدام Anchors أو ملفات منفصلة قابلة للدمج.

أفضل الممارسات

أفضل الممارسات

استخدم JSON للتواصل بين الخدمات عبر الشبكة، وYAML لإعداد البنية التحتية والـ CI/CD اللي هيقرأها ويعدّلها بني آدمين.

أفضل الممارسات

افصل إعدادات كل بيئة (Development/Testing/Production) في ملفات منفصلة أو باستخدام Anchors بدل تكرار نفس القيم في كل مكان.

ملخص الفصل

- JSON هي العمود الفقري لتبادل البيانات في REST APIs وGraphQL وMicroservices.
- YAML هي المعيار الفعلي لملفات الإعداد في Docker وKubernetes وCI/CD وMLOps.
- اختيار الصيغة المناسبة يعتمد على طبيعة الاستخدام: تبادل بيانات سريع بين آلات (JSON) أم إعداد يقرأه ويعدّله بشر (YAML).

تمارين عملية

تمرين ١: اكتب ملف docker-compose.yml بسيط لخدمة Python API متصلة بقاعدة بيانات Redis.

الحل المقترح

services: api: build: . ports: - "8000:8000" depends_on: - redis redis: image: redis:7

الفصل السابع — الأداء — Performance

لما البيانات بتكبر والحمل بيزيد، الفروق الصغيرة في الأداء بتتحول لفروق كبيرة جدًا. الفصل ده هنقارن فيه الأداء الفعلي بين JSON وYAML من زوايا مختلفة.

Parsing Performance

JSON أسرع بشكل واضح جدًا في الـ Parsing من YAML، والسبب بسيط: قواعد JSON محدودة وصريحة (أقواس، فواصل)، فالـ Parser بيقدر يتحرك بسرعة من غير الحاجة لتتبع مستويات Indentation أو حل Anchors وAliases. YAML على الجانب التاني بتحتاج معالجة أعقد بكتير: تتبع المسافات بدقة، بناء جدول مرجعي للـ Anchors، التعامل مع أساليب الكتابة المختلفة (Block/Flow).

> **ملاحظة**
>
> في اختبارات معيارية شائعة (Benchmarks) على نفس البيانات، PyYAML بيكون أبطأ من json module الأساسية في Python بعدة أضعاف (غالبًا من ٥ إلى ١٠ أضعاف حسب حجم وتعقيد الملف).

Serialization Speed

نفس المنطق بينطبق على عملية التحويل العكسية (تحويل كائن Python لنص): json.dumps() أسرع بكتير من yaml.dump()، خصوصًا مع بيانات كبيرة الحجم أو متداخلة بعمق.

File Size

JSON غالبًا بتنتج حجم ملف أصغر من YAML لنفس البيانات، خصوصًا لو استخدمت separators مضغوطة وشلت الـ indent؛ YAML بتحتاج مسافات إضافية لتمثيل التداخل، وده بيزود حجم الملف نسبيًا. الفرق ده بسيط في الملفات الصغيرة لكن بيبان في البيانات الضخمة.

Memory Usage

استهلاك الذاكرة أثناء التحليل بيعتمد على حجم الشجرة الوسيطة اللي الـ Parser بيبنيها؛ YAML عمومًا بتستهلك ذاكرة أكتر شوية بسبب تعقيد الـ Grammar والحاجة لتتبع الـ Anchors في جدول مرجعي إضافي طوال عملية الـ Parsing.

Compression

كل من JSON وYAML صيغ نصية (Text-based)، فبتستفيد بشكل كبير جدًا من ضغط عام زي gzip أو Brotli قبل النقل عبر الشبكة. عمليًا، أغلب REST APIs الكبيرة بتضغط استجابات JSON بـ gzip تلقائيًا، وده بيقلل الفرق في الحجم بين الصيغتين إلى حد كبير.

Streaming

لو عندك ملف JSON ضخم جدًا (أكبر من الذاكرة المتاحة)، مش لازم تحمّله كله مرة واحدة؛ فيه مكتبات Streaming JSON Parsers (زي ijson في Python) بتقرأ الملف تدريجيًا عنصر بعنصر بدل تحميله بالكامل. YAML أضعف بكتير في المجال ده بسبب اعتمادها على السياق الكامل للمستند (خصوصًا مع Anchors) لحل المراجع بشكل صحيح.

Optimization Techniques

- استخدم مكتبات JSON مبنية بلغة أسرع من Python نفسها زي orjson أو ujson لو الأداء عامل حرج.
- قلل عمق التداخل (Nesting) قدر الإمكان في البيانات الضخمة المتكررة.
- استخدم separators مضغوطة وdsh الـ indent في JSON المرسلة عبر الشبكة (مش الملفات المخصصة للقراءة البشرية).
- للـ YAML، لو الأداء مهم، فكّر في تحويل الإعداد لـ JSON أثناء الإنتاج (Production) والاحتفاظ بـ YAML بس في مرحلة التطوير للقراءة الأسهل.

مقارنة الأداء: JSON مقابل YAML

المعيار	JSON	YAML
سرعة الـ Parsing	سريعة جدًا	أبطأ بعدة أضعاف
سرعة الـ Serialization	سريعة جدًا	أبطأ نسبيًا
حجم الملف (بدون ضغط)	أصغر عادةً	أكبر بسبب المسافات
استهلاك الذاكرة أثناء التحليل	أقل	أعلى قليلًا
دعم Streaming	متوفر (ijson وغيرها)	محدود جدًا
الأنسب للأداء العالي	✅ نعم	غير موصى به

مثال عملي: قياس الفرق بنفسك (Benchmark بسيط)

بدل ما تصدّق الأرقام العامة، جرّب بنفسك على جهازك بكود بسيط زي ده، وقارن النتيجة على بياناتك الفعلية:

```
import json, yaml, time

data = {"items": [{"id": i, "name": f"item_{i}"} for i in range(10000)]}

start = time.perf_counter()
json_text = json.dumps(data)
print("JSON dumps:", time.perf_counter() - start)

start = time.perf_counter()
yaml_text = yaml.dump(data)
print("YAML dump:", time.perf_counter() - start)

start = time.perf_counter()
json.loads(json_text)
print("JSON loads:", time.perf_counter() - start)

start = time.perf_counter()
yaml.safe_load(yaml_text)
print("YAML safe_load:", time.perf_counter() - start)
```

على بيانات بالحجم ده، الفرق بين JSON وYAML عادةً بيبان بوضوح شديد — YAML ممكن تاخد وقت أطول بعشرات المرات في بعض الحالات، وده بيوضح عمليًا ليه JSON هي الخيار الافتراضي لأي مسار حرج للأداء.

الأسئلة الشائعة

❓ لو JSON أسرع بكتير، ليه ما بنستبدلش YAML بيها في كل حاجة؟

لأن الأداء مش دايمًا العامل الحاسم؛ ملفات الإعداد بتتقرأ وتتحلل مرة واحدة عند بدء التشغيل (Startup) وقلما تتكرر آلاف المرات في الثانية، فالفرق في الأداء غير محسوس عمليًا، بينما فايدة القراءة البشرية والتعليقات بتفضل قيمة عالية طول الوقت.

❓ هل orjson أسرع من json module الأساسية بشكل ملحوظ؟

نعم، orjson مكتوبة بلغة Rust وبتكون أسرع بعدة أضعاف من json module القياسية في Python، خصوصًا مع بيانات ضخمة، وبتُستخدم كتير في خدمات الإنتاج عالية الحمل.

أخطاء شائعة

⚠ خطأ شائع

استخدام YAML لتبادل بيانات API بحجم كبير عبر الشبكة ظنًا إن الفرق في الأداء مش مهم — في أنظمة عالية الحمل (High Throughput)، الفرق بيتراكم ويبقى له أثر حقيقي.

⚠ خطأ شائع

تحميل ملف JSON ضخم جدًا بالكامل في الذاكرة بدل استخدام Streaming Parser، وده بيسبب مشاكل نفاد الذاكرة (Out of Memory) في البيانات الكبيرة جدًا.

أفضل الممارسات

☑ أفضل الممارسات

استخدم JSON دايمًا في أي مسار حرج للأداء (Hot Path) زي استجابات API عالية التردد.

☑ أفضل الممارسات

استخدم أدوات Benchmark فعلية (مش افتراضات) قبل ما تقرر تغيير صيغة البيانات في نظام موجود بالفعل، لأن التأثير الحقيقي بيختلف حسب حجم وطبيعة البيانات.

ملخص الفصل

- JSON أسرع بشكل واضح من YAML في كل من الـ Parsing والـ Serialization، بسبب بساطة قواعدها مقارنة بتعقيد مواصفة YAML.
- الفرق في الأداء بيبقى مهم جدًا في المسارات الحرجة (APIs عالية الحمل) لكنه غير محسوس عمليًا في ملفات الإعداد اللي بتتحلل مرة واحدة عند التشغيل.
- لتحسين الأداء أكتر، فكّر في مكتبات JSON مبنية بلغات أسرع (orjson) واستخدم Streaming للملفات الضخمة جدًا.

تمارين عملية

تمرين ١: ليه استهلاك الذاكرة في تحليل YAML أعلى من JSON رغم إن الاتنين بيمثلوا نفس البيانات؟

💡 الحل المقترح

لأن YAML Parser محتاج يحتفظ بجدول مرجعي إضافي (Reference Table) لحل الـ Anchors والـ Aliases أثناء التحليل، بالإضافة لتعقيد تتبع مستويات الإزاحة، وده بيزود الحمل على الذاكرة مقارنة بـ JSON البسيطة.

الفصل الثامن — Debugging — تتبع وحل الأخطاء

مفيش مبرمج ما واجهش رسالة خطأ غامضة من Parser JSON أو YAML وضاع وقت كبير يدور على مكان المشكلة. الفصل ده هيديك أدوات وطريقة تفكير منظمة لحل المشاكل دي بسرعة.

Parsing Errors في JSON

رسائل خطأ JSON بتكون واضحة نسبيًا لأن الـ Syntax محدود؛ أشهر الأخطاء:

```
>>> import json
>>> json.loads('{"name": "Ahmed",}')
json.decoder.JSONDecodeError: Expecting property name enclosed in double quotes:
line 1 column 19 (char 18)
```

الرسالة هنا واضحة: فيه فاصلة زيادة قبل القوس المغلق. الرسالة بتديك line وcolumn بالظبط، وده بيسهّل الوصول للمشكلة فورًا.

أشهر أخطاء JSON

- Expecting ',' delimiter: غالبًا فاصلة ناقصة بين عنصرين.
- Expecting property name enclosed in double quotes: مفتاح بدون علامتي تنصيص، أو فاصلة زيادة قبل القوس المغلق.
- Extra data: فيه محتوى زيادة بعد نهاية الـ JSON الصحيح (زي نسخ ملفين في واحد بالغلط).
- Unterminated string: علامة تنصيص اتفتحت ومتقفلتش.

Parsing Errors في YAML

رسائل خطأ YAML عادةً أطول وأعقد شوية، لكنها كمان بتوضح مكان المشكلة بالسطر والعمود:

```
>>> import yaml
>>> yaml.safe_load("""
... name: Ahmed
...   age: 30
... """)
yaml.scanner.ScannerError: mapping values are not allowed here
  in "<unicode string>", line 3, column 6
```

Indentation Problems

أشهر مصدر لمشاكل YAML على الإطلاق. علامات التحذير الشائعة:

- mapping values are not allowed here: غالبًا سطر فيه إزاحة زيادة أو ناقصة غير متوقعة.
- could not find expected ':': مفتاح من غير نقطتين، أو مشكلة في مستوى الإزاحة أدت لتفسير خاطئ للسطر.
- found character '\t' that cannot start any token: استخدام Tab بدل Spaces — YAML بترفضها تمامًا.

Encoding Issues

لما بيظهر رموز غريبة (Mojibake) بدل النص العربي أو رموز خاصة، غالبًا السبب فتح الملف بترميز مختلف عن اللي اتكتب بيه أصلًا. الحل: حدد encoding='utf-8' صراحة دايمًا في كل عمليات فتح الملفات.

Validation Errors

دي أخطاء بتحصل بعد نجاح الـ Parsing، لما البيانات صحيحة Syntax لكن غير متوافقة مع الـ Schema أو الـ Model المتوقع (زي مفتاح ناقص أو نوع بيانات خاطئ). رسائل الخطأ في jsonschema أو Pydantic عادةً بتوضح بالظبط أي حقل فشل ولية.

أدوات Debugging (Debugging Tools)

- JSON Linters/Validators أونلاين (مع الحرص على عدم رفع بيانات حساسة عليها).
- yamllint: أداة سطر أوامر بتفحص ملفات YAML وبتطلع تحذيرات عن الإزاحة والتنسيق قبل ما تشغّل الملف فعليًا.
- VS Code Extensions: دعم مدمج لتلوين الأخطاء (Syntax Highlighting) وإظهار المشاكل مباشرة أثناء الكتابة لكل من JSON وYAML.
- python -m json.tool: أداة مدمجة في Python بتاخد ملف JSON وتتحقق من صحته وتنسّقه، مفيدة جدًا للفحص السريع من الـ Terminal.

```
cat config.json | python -m json.tool
```

استراتيجية عامة لحل مشاكل Parsing

١. اقرأ رسالة الخطأ بالكامل — الـ line وcolumn عادةً موجودين وبيوجهوك للمكان بالظبط.

٢. لو الملف كبير، قسّمه لأجزاء أصغر واختبر كل جزء لوحده لعزل مكان المشكلة.

٣. استخدم أداة Linter قبل ما تشغّل الكود، مش بعده.

٤. تأكد من الـ Encoding والمسافات (خصوصًا في YAML) قبل أي حاجة تانية.

الأسئلة الشائعة

؟ ليه رسائل خطأ YAML أحيانًا بتشاور على سطر غلط تمامًا؟

لأن الـ Parser بيكتشف المشكلة فعليًا في السطر اللي بعد السطر اللي فيه الخطأ الحقيقي، لأنه بيكون لسه بيحاول يفهم المستند حتى تلك اللحظة؛ لو السطر المذكور شكله سليم، افحص السطر أو السطرين اللي قبله كمان.

؟ هل فيه فرق بين JSONDecodeError وValidationError؟

نعم؛ JSONDecodeError معناها إن النص نفسه مش JSON صحيح من الأساس (مشكلة Syntax)، أما ValidationError فمعناها إن النص اتحلل بنجاح لكن البيانات جواه مش متوافقة مع الـ Schema أو الـ Model المتوقع.

أخطاء شائعة

⚠ خطأ شائع

تجاهل رسالة الخطأ الكاملة والذهاب مباشرة لتخمين المشكلة — رسائل JSON وYAML عادةً دقيقة جدًا في تحديد المكان.

⚠ خطأ شائع

استخدام Print وTrial-and-Error بدل استخدام أداة Linter مخصصة توفر وقت كبير.

أفضل الممارسات

> ☑ **أفضل الممارسات**
>
> دمج yamllint وjson schema validators في الـ Pre-commit Hooks أو الـ CI Pipeline، عشان تمسك الأخطاء قبل ما توصل للإنتاج.

> ☑ **أفضل الممارسات**
>
> لو بتكتب ملف كبير يدويًا، اكتبه واختبره على أجزاء صغيرة بدل كتابته كامل ثم اختباره مرة واحدة.

ملخص الفصل

- رسائل خطأ JSON وYAML بتوضح line وcolumn عادةً، وده بيسهّل تحديد مكان المشكلة بدقة.
- أشهر مشاكل YAML مرتبطة بالإزاحة (Indentation) والخلط بين Tabs وSpaces.
- الـ Validation Errors مختلفة عن الـ Parsing Errors: الأولى بتحصل بعد نجاح التحليل ومرتبطة بمنطق التطبيق.

تمارين عملية

تمرين ١: الكود ده بيرمي خطأ: yaml.safe_load('key: value\n\tkey2: value2') — إيه سبب المشكلة وإزاي تحلها؟

> 💡 **الحل المقترح**
>
> المشكلة إن السطر التاني بيستخدم Tab للإزاحة بدل Spaces، وYAML بترفض الـ Tab تمامًا. الحل: استبدل الـ Tab بعدد مناسب من الـ Spaces.

الفصل التاسع — Security — الأمان

الفصل ده من أهم فصول الكتاب كله، لأن التعامل الخاطئ مع YAML تحديدًا سبّب ثغرات أمنية حقيقية في مشاريع كبيرة على مدار السنين. هنشرح المشكلة بعمق من غير ما نديك أي كود فعلي لاستغلالها — الهدف فهم المخاطر وطرق الحماية بس.

Secure Parsing — المبدأ العام

المبدأ الذهبي في أي نظام بيتعامل مع بيانات خارجية: لا تثق أبدًا في البيانات القادمة من مصدر غير موثوق (Untrusted Input) من غير تحقق وحماية كافية. ده بينطبق بشكل خاص على YAML بسبب مرونتها الزيادة في تمثيل أنواع بيانات معقدة.

Unsafe Loaders — ليه YAML خطيرة لو استخدمتها غلط

المشكلة الجوهرية إن بعض مكتبات YAML (وأشهرها PyYAML قديمًا) بتقدر — لو استخدمت الـ Loader الخاطئ — تبني أي كائن Python من الملف، مش بس أنواع البيانات البسيطة (Dict, List, String). ده معناه إن ملف YAML خبيث ممكن يستغل الـ Tags الخاصة (زي !!python/object/apply) عشان يخلي الـ Parser "ينفّذ" كود Python فعلي أثناء عملية القراءة نفسها، مش بعدها.

> **ملاحظة**
>
> الفرق الجوهري عن JSON: JSON Parser مبدئيًا مايقدرش "ينفّذ" أي كود إطلاقًا مهما كانت محتويات الملف، لأن مواصفتها بتسمح فقط بأنواع بيانات بسيطة صريحة. أما YAML القديمة (بالـ Loader الخاطئ) فبتفتح الباب لتنفيذ كود عشوائي، وده فرق جوهري في مستوى الخطورة بين الصيغتين.

Trusted مقابل Untrusted Data

- Trusted Data: ملفات إعداد كتبها فريقك بنفسه ومخزّنة في الـ Repository الخاص بالمشروع تحت مراجعة (Code Review).
- Untrusted Data: أي بيانات جاية من مستخدم خارجي، API عام، رفع ملفات، أو أي مصدر مش تحت سيطرتك الكاملة.

القاعدة العملية: تعامل مع كل بيانات YAML وكأنها Untrusted افتراضيًا، حتى لو حاسس إنها Trusted، إلا لو عندك سبب موثّق وواضح يبرر عكس ده.

الحماية الصحيحة

- استخدم دايمًا yaml.safe_load() بدل yaml.load() في PyYAML — ده بيحمّل بس أنواع البيانات البسيطة والآمنة.
- في نسخ PyYAML الحديثة (٥٫١+)، حتى yaml.load() العادي بقى بيطلب منك تحدد Loader صراحة، وده تحسين أمني مهم قدمه المشروع نفسه بعد اكتشاف المشكلة دي على نطاق واسع.
- في ruamel.yaml، استخدم YAML(typ='safe') للحصول على نفس مستوى الحماية.
- لا تعتمد أبدًا على FullLoader أو UnsafeLoader إلا لو كنت متحكم بالكامل في مصدر الملف، وموثّق سبب الحاجة لده صراحة في الكود.

```
import yaml

# آمن دايمًا – بيحمّل بس أنواع بيانات بسيطة
data = yaml.safe_load(untrusted_input)

# خطر لو الملف من مصدر غير موثوق
# data = yaml.load(untrusted_input, Loader=yaml.UnsafeLoader)
```

Code Injection وYAML Exploits — نظرة عامة على طبيعة المشكلة

من غير الدخول في تفاصيل تقنية قابلة للاستغلال، المبدأ العام إن أي صيغة بتدعم "Tags مخصصة" أو "بناء كائنات تلقائي" بتحتاج حذر إضافي، لأن الميزة اللي بتدّي مرونة للمطور (بناء كائنات Python مخصصة تلقائيًا من الملف) هي نفسها اللي بتقدر تُستغل لو الملف جاي من مصدر خبيث. الحل مش تجنب الميزة كليًا، لكن حصرها في السياقات الموثوقة بس، واستخدام الوضع الآمن (Safe Mode) كافتراضي في كل حاجة تانية.

Common Vulnerabilities — أنواع عامة

- Arbitrary Code Execution: تنفيذ كود غير مقصود بسبب Loader غير آمن (المشكلة الأشهر في YAML تحديدًا).
- Denial of Service (Billion Laughs / Entity Expansion): ملفات مصممة بذكاء باستخدام Anchors وAliases متداخلة بشكل كبير جدًا، بحيث تتوسع القيمة الفعلية عند التحليل لحجم هائل يستهلك كل الذاكرة المتاحة ويوقف الخدمة.
- Resource Exhaustion عبر ملفات JSON أو YAML ضخمة جدًا أو متداخلة بعمق غير محدود، وده بيأثر على أي صيغة مش YAML بس.

> ⚠ **خطأ شائع**
> قبول رفع ملفات YAML من مستخدمين خارجيين (Public Upload) وتحليلها مباشرة بدون Loader آمن ولا حدود على حجم أو عمق التداخل.

الحماية من استنزاف الموارد (Resource Exhaustion)

بعيدًا عن تنفيذ الكود، فيه فئة تانية من المخاطر بتنطبق على JSON وYAML مع بعض: ملفات مصممة بحجم أو تعقيد كبير قصده استهلاك موارد السيرفر (Memory/CPU) وإيقاف الخدمة (Denial of Service)، حتى لو الملف نفسه "آمن" من ناحية عدم تنفيذ كود.

- ضع حد أقصى صريح لحجم أي ملف مرفوع أو مستقبل عبر API (زي ١MB أو ١٠MB حسب طبيعة النظام).
- ضع حد أقصى لعمق التداخل (Nesting Depth) المسموح به، لأن بعض الـ Parsers بتنهار (Stack Overflow) مع تداخل عميق جدًا وغير طبيعي.
- في YAML تحديدًا، انتبه لعدد الـ Anchors والـ Aliases المسموح بيه في ملف واحد، لأن تداخلها بذكاء ممكن يخلي حجم البيانات الفعلي بعد التوسّع أكبر بكتير من حجم الملف الخام على القرص.

```
import json

MAX_SIZE = 1_000_000  # 1 MB

def safe_load_json(raw_bytes):
    if len(raw_bytes) > MAX_SIZE:
        raise ValueError("File too large")
    return json.loads(raw_bytes)
```

Best Security Practices — ملخص عملي

أفضل الممارسات

استخدم safe_load دايمًا كافتراضي، ولا تغيّره إلا بسبب موثّق وواضح.

أفضل الممارسات

ضع حدود واضحة على حجم الملف المسموح برفعه، وعمق التداخل المسموح به، خصوصًا في أي نقطة بترفع فيها ملفات من مستخدمين خارجيين.

أفضل الممارسات

راجع بانتظام إصدارات مكتبات JSON وYAML المستخدمة في مشروعك، وحدّثها فورًا لو صدرت تحديثات أمنية (Security Patches).

أفضل الممارسات

لا تخزّن معلومات حساسة (كلمات مرور، مفاتيح API) مباشرة في ملفات YAML أو JSON عادية داخل الـ Repository؛ استخدم أدوات إدارة أسرار مخصصة (Secrets Manager, Vault, Kubernetes Secrets).

الأسئلة الشائعة

هل JSON عندها نفس مشكلة تنفيذ الكود اللي عند YAML؟

لا، مواصفة JSON نفسها مايفيهاش أي آلية لبناء كائنات مخصصة أو تنفيذ كود أثناء الـ Parsing، فالخطر ده غير موجود بنيويًا في JSON. المخاطر المرتبطة بـ JSON بتكون غالبًا في مستوى التطبيق نفسه (زي ثغرات في الكود اللي بيعالج البيانات بعد التحليل)، مش في الـ Parser ذاته.

هل safe_load كافية لحماية كاملة من كل المخاطر؟

safe_load بتحمي من تنفيذ الكود العشوائي، لكنها لوحدها مش كافية ضد هجمات استنزاف الموارد (زي Anchors متداخلة بشكل كبير)؛ لازم كمان تضع حدود صريحة على حجم وعمق الملفات المقبولة.

أخطاء شائعة

خطأ شائع

الاعتماد على PyYAML بنسخة قديمة جدًا (قبل ٥,١) بدون تحديث، وده بيسيب النظام معرّض للثغرة الأشهر في تاريخ المكتبة.

خطأ شائع

افتراض إن ملف YAML "من داخل الشركة" آمن تلقائيًا بدون أي تحقق إضافي.

أفضل الممارسات

> ☑ **أفضل الممارسات**
>
> اجعل safe_load وSafeLoader هما الخيار الافتراضي المكتوب في كل الكود الجديد، وراجع الكود القديم للتأكد من عدم وجود Loaders غير آمنة متروكة من غير داعي.

ملخص الفصل

- الخطر الأمني الأكبر في YAML هو استخدام Loaders غير آمنة اللي بتقدر تنفّذ كود عشوائي من ملف خبيث.
- الحل الأساسي والدائم: استخدم safe_load / SafeLoader دايمًا كافتراضي في أي مشروع.
- JSON لا تعاني من نفس المشكلة بنيويًا، لكن كل الصيغ عرضة لهجمات استنزاف الموارد لو مفيش حدود على حجم وعمق الملفات المقبولة.
- أسرار المشروع (كلمات مرور، مفاتيح) ما تتخزنش أبدًا كنص عادي داخل ملفات JSON/YAML في الـ Repository.

تمارين عملية

تمرين ١: ليه yaml.safe_load() أفضل من yaml.load() حتى لو الملف من مصدر تعتبره موثوق؟

> 💡 **الحل المقترح**
>
> لأن 'موثوق' مش ضمان دائم؛ الملف ممكن يتعدل بالخطأ، أو يتسرب منه محتوى ضار عبر نظام آخر متصل بمصدر الملف، والعادة الآمنة الدائمة بتحميك من مفاجآت مستقبلية بغض النظر عن الوضع الحالي.

الفصل العاشر — Best Practices — أفضل الممارسات

الفصل ده تجميع منظّم لأفضل الممارسات اللي اتوزعت في الفصول السابقة، مع إضافات جديدة عن التنظيم والصيانة على المدى الطويل.

Naming Convention

- التزم بأسلوب تسمية واحد ثابت للمفاتيح في كل المشروع: naming_case (snake_case) شائع في Python/Backend، أما camelCase شائع في JavaScript/Frontend.
- لا تخلط بين الأسلوبين في نفس الملف أو المشروع الواحد؛ الاختيار الأهم هو الاتساق، مش أي أسلوب تحديدًا.
- أسماء الملفات نفسها لازم تكون واضحة ومعبّرة عن الغرض: config.yaml أفضل من settings1.yaml.

File Organization

- افصل إعدادات كل بيئة (Development/Testing/Production) في ملفات منفصلة: config.dev.yaml، config.prod.yaml.
- استخدم مجلد config/ مركزي بدل توزيع ملفات الإعداد في أماكن عشوائية داخل المشروع.
- افصل الأسرار (Secrets) عن الإعدادات العامة تمامًا، حتى لو كانا بنفس الصيغة.

Maintainability و Readability

- في YAML، استخدم تعليقات (#) لتوضيح سبب أي قيمة غير بديهية، مش بس "إيه" هي القيمة لكن "ليه" هي كده.
- في JSON، لو محتاج توثيق، ضيف حقل خاص زي "_comment" أو استخدم ملف Markdown منفصل بجانب ملف الإعداد.
- قسّم الملفات الكبيرة جدًا (أكتر من بضع مئات الأسطر) لملفات أصغر مترابطة بدل ملف واحد ضخم يصعب تصفحه.

Documentation

أي ملف Configuration مهم لازم يترافق معاه توثيق واضح — README قريب منه، أو تعليقات مباشرة داخل الملف نفسه (في حالة YAML) توضح: إيه الغرض من كل قسم؟ إيه القيم المسموحة؟ إيه اللي بيحصل لو اتغيرت القيمة دي؟

Version Control

- خزّن ملفات الإعداد (غير الحساسة) في Git زي أي كود آخر، عشان تتبع تاريخ التغييرات.
- لا تخزّن أبدًا ملفات فيها أسرار حقيقية (كلمات مرور، مفاتيح API) في Git، حتى في Repository خاص؛ استخدم gitignore. واستبدلها بملفات .example فاضية من القيم الحقيقية.

```
# .gitignore
config.prod.yaml
secrets.json

# بدلًا منه، خزّن:
config.prod.yaml.example
```

Reusability

- استخدم Anchors وAliases في YAML لتقليل التكرار بين البيئات المتشابهة.
- في JSON، لو محتاج إعادة استخدام هيكل، فكّر في استخدام JSON Schema مع $ref، أو توليد الملف برمجيًا من قالب (Template) بدل النسخ اليدوي.

Formatting Standards

- استخدم أدوات Formatting تلقائية (Prettier لـ JSON، yamlfmt أو Prettier لـ YAML) ودمجها في الـ Pre-commit Hooks لضمان اتساق التنسيق بين كل أعضاء الفريق.
- اتفق كفريق على عدد مسافات الإزاحة الموحد في YAML (الأشهر: مسافتين) وثبّته في إعدادات الـ Editor المشتركة (.editorconfig).

> ☑ **أفضل الممارسات**
>
> استخدم ملف .editorconfig موحّد في جذر المشروع يحدد نوع الإزاحة (Spaces) وعددها لكل أنواع الملفات، عشان يضمن اتساق التنسيق بغض النظر عن الـ Editor اللي كل عضو في الفريق بيستخدمه.

قائمة مراجعة سريعة (Checklist) قبل أي Commit

البند	تحقق منه
الاتساق في التسمية	أسلوب واحد ثابت (snake_case أو camelCase) في كل الملف
الأسرار	لا يوجد كلمات مرور أو مفاتيح API داخل الملف
التنسيق	الملف مرّ على Formatter/Linter قبل الحفظ
التعليقات (YAML)	أي قيمة غير بديهية موثقة بتعليق واضح
البيئات	الفروقات بين البيئات فى ملفات Override منفصلة، مش مكررة
الحجم	الملف مش أكبر من اللازم؛ مقسّم لو تعدى بضع مئات الأسطر

الأسئلة الشائعة

❓ هل الأفضل أستخدم snake_case ولا camelCase في ملفات JSON؟

مفيش إجابة صحيحة مطلقة؛ القاعدة الوحيدة الثابتة هي الاتساق. لو الـ API بترجع بيانات لـ Frontend مكتوب بـ JavaScript، camelCase أكتر طبيعية هناك، لكن لو الملف داخلي بالكامل في مشروع Python، snake_case هيبقى أكتر انسجامًا مع باقي الكود.

أخطاء شائعة

⚠ خطأ شائع

خلط أسلوب snake_case وcamelCase في نفس ملف الإعداد بدون سبب واضح، وده بيصعّب القراءة ويزود احتمال الأخطاء.

⚠ خطأ شائع

رفع ملف إعداد فيه كلمة مرور حقيقية بالغلط على Git، حتى لو اتحذف بعدين — البيانات بتفضل موجودة في تاريخ الـ Commits القديمة إلا لو تم تنظيف التاريخ صراحة.

أفضل الممارسات — ملخص شامل

☑ أفضل الممارسات

ثبّت أسلوب تسمية واحد، نظّم الملفات في مجلد مركزي، وثّق كل قيمة غير بديهية، وافصل الأسرار عن الإعدادات العامة تمامًا.

ملخص الفصل

- الاتساق في التسمية والتنسيق أهم من اختيار أسلوب معين بذاته.
- الأسرار الحقيقية ما تتخزنش أبدًا في ملفات إعداد عادية داخل Git.
- أدوات Formatting وLinting الآلية بتوفر وقت كبير وتقلل الأخطاء البشرية على المدى الطويل.

تمارين عملية

تمرين ١: اقترح هيكل مجلدات لمشروع Python فيه إعدادات لثلاث بيئات مختلفة (dev, test, prod) مع فصل الأسرار.

💡 الحل المقترح

```
config/
  base.yaml
  dev.yaml
  test.yaml
  prod.yaml
  secrets/
) يستثني كل شيء في هذا المجلد .gitignore (
    prod.secrets.yaml.example
```

الفصل الحادي عشر — مشاريع عملية

في الفصل ده هنبني مشاريع صغيرة كاملة خطوة بخطوة، الهدف إنك تطبق كل حاجة اتعلمناها في سياق واقعي.

مشاريع JSON

مشروع ١: إعداد ملف Configuration لتطبيق Backend

1. حدد الأقسام الأساسية: database, server, logging.
2. اكتب الملف بحيث يكون كل قسم Object منفصل بمفاتيح واضحة.
3. حمّل الملف في التطبيق باستخدام json.load() عند بدء التشغيل، مش أثناء التنفيذ.

```
{
  "server": { "host": "0.0.0.0", "port": 8000, "debug": false },
  "database": { "engine": "postgresql", "host": "localhost", "port": 5432, "name":
"app_db" },
  "logging": { "level": "INFO", "format": "json" }
}
```

مشروع ٢: تحويل CSV إلى JSON

4. اقرأ ملف CSV سطر بسطر باستخدام csv.DictReader.
5. خزّن كل سطر كـ Dictionary داخل List واحدة.
6. اكتب الناتج كملف JSON منسّق.

```
import csv, json

with open('data.csv', encoding='utf-8') as f:
    reader = csv.DictReader(f)
    rows = list(reader)

with open('data.json', 'w', encoding='utf-8') as f:
    json.dump(rows, f, indent=2, ensure_ascii=False)
```

مشروع ٣: تحويل JSON إلى CSV

7. اقرأ ملف JSON فيه List of Objects بنفس المفاتيح.
8. استخرج أسماء الأعمدة من مفاتيح أول عنصر.
9. اكتب البيانات باستخدام csv.DictWriter.

```
import csv, json

with open('data.json', encoding='utf-8') as f:
    rows = json.load(f)

with open('data.csv', 'w', newline='', encoding='utf-8') as f:
    writer = csv.DictWriter(f, fieldnames=rows[0].keys())
```

```
    writer.writeheader()
    writer.writerows(rows)
```

مشروع ٤: إعداد ملفات Localization (ترجمة الواجهات)

10. اعمل ملف منفصل لكل لغة: ar.json، en.json.

11. استخدم نفس المفاتيح بالظبط في كل الملفات، والقيمة بس هي اللي بتختلف.

```
// ar.json
{ "welcome_message": "أهلاً بيك", "logout_button": "تسجيل الخروج" }

// en.json
{ "welcome_message": "Welcome", "logout_button": "Logout" }
```

☑ أفضل الممارسات

استخدم أداة للتحقق من إن كل ملفات اللغات فيها نفس المفاتيح بالظبط، لتجنب ظهور مفتاح ناقص في لغة معينة أثناء التشغيل الفعلي.

مشروع ٥: إعداد REST API Response موحّد

12. عرّف شكل موحّد لكل استجابات الـ API (Envelope) بحيث كل استجابة فيها نفس الحقول الأساسية.

13. استخدم status وdata وmeta وerrors كحقول ثابتة، سواء نجح الطلب أو فشل.

```
// استجابة ناجحة
{
  "status": "success",
  "data": { "id": 1, "name": "Ahmed" },
  "meta": { "requestId": "abc-123" }
}

// استجابة فاشلة
{
  "status": "error",
  "errors": [ { "field": "email", "message": "Invalid email format" } ],
  "meta": { "requestId": "abc-124" }
}
```

☑ أفضل الممارسات

ثبّت شكل الاستجابة (Envelope) ده في كل الـ Endpoints بالمشروع، عشان الـ Frontend يقدر يتعامل مع كل الاستجابات بنفس المنطق الموحّد بدون حالات خاصة.

مشروع ٦: تحويل JSON إلى Excel

14. اقرأ ملف JSON فيه List of Objects.

15. حوّله لـ DataFrame باستخدام pandas.

16. اكتبه كملف Excel باستخدام to_excel().

```
import pandas as pd
import json

with open('data.json', encoding='utf-8') as f:
    rows = json.load(f)

df = pd.DataFrame(rows)
df.to_excel('data.xlsx', index=False, engine='openpyxl')
```

مشروع ٧: تخزين بيانات تطبيق محلي (Local App Data Storage)

17. استخدم ملف JSON بسيط كقاعدة بيانات محلية لتطبيق صغير (زي To-Do List).

18. اقرأ الملف عند بدء التشغيل، وعدّل عليه في الذاكرة، واكتبه تاني بعد كل تعديل.

```
import json
from pathlib import Path

DB_PATH = Path('todos.json')

def load_todos():
    if not DB_PATH.exists():
        return []
    return json.loads(DB_PATH.read_text(encoding="utf-8"))

def save_todos(todos):
    DB_PATH.write_text(json.dumps(todos, indent=2, ensure_ascii=False),
encoding="utf-8")

todos = load_todos()
todos.append({"task": "اكتب الفصل الحادي عشر", "done": False})
save_todos(todos)
```

> ⚠ **خطأ شائع**
>
> استخدام ملف JSON كقاعدة بيانات لتطبيق فيه كتابة متزامنة من أكتر من مستخدم في نفس الوقت — ده بيسبب فقدان بيانات (Race Condition)؛ الحل المناسب لهذه الحالة هو قاعدة بيانات حقيقية (SQLite على الأقل) مش ملف JSON.

مشروع ٨: ملفات Settings منفصلة لكل بيئة

19. اعمل ملف settings.base.json فيه القيم المشتركة.

20. اعمل settings.dev.json وsettings.prod.json فيهم بس الفروقات، وادمجهم برمجيًا وقت التشغيل.

```
import json

def load_settings(env):
    with open('settings.base.json', encoding='utf-8') as f:
        settings = json.load(f)
    with open(f'settings.{env}.json', encoding='utf-8') as f:
        overrides = json.load(f)
    settings.update(overrides)
    return settings
```

```
config = load_settings('prod')
```

مشاريع YAML

مشروع ٩: Docker Compose لتطبيق ويب كامل

21. عرّف خدمة الـ API، وخدمة قاعدة البيانات، وخدمة الكاش (Redis).
22. استخدم depends_on لضمان ترتيب التشغيل الصحيح.
23. استخدم environment لتمرير إعدادات الاتصال بين الخدمات.

```
services:
  api:
    build: .
    ports: ["8000:8000"]
    environment:
      - DATABASE_URL=postgresql://db:5432/app
      - REDIS_URL=redis://cache:6379
    depends_on: [db, cache]
  db:
    image: postgres:16
  cache:
    image: redis:7
```

مشروع ١٠: Kubernetes Manifest بسيط لخدمة API

24. عرّف Deployment بعدد Replicas مناسب.
25. عرّف Service لتوجيه الحركة (Traffic) للـ Pods.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api-deployment
spec:
  replicas: 2
  selector:
    matchLabels: { app: api }
  template:
    metadata:
      labels: { app: api }
    spec:
      containers:
        - name: api
          image: myapp/api:latest
          ports: [{ containerPort: 8000 }]
---
apiVersion: v1
kind: Service
metadata:
  name: api-service
spec:
  selector: { app: api }
  ports:
    - port: 80
      targetPort: 8000
```

مشروع ١١: GitHub Actions Workflow لاختبار وبناء مشروع Python

26. شغّل الاختبارات على كل push وpull_request.
27. استخدم Matrix لاختبار أكثر من نسخة Python في نفس الوقت.

```yaml
name: CI
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        python-version: ["3.10", "3.11", "3.12"]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "${{ matrix.python-version }}" }
      - run: pip install -r requirements.txt
      - run: pytest
```

مشروع ١٢: ملف إعداد Machine Learning (Hyperparameters)

28. عرّف إعدادات النموذج والتدريب في أقسام منفصلة وواضحة.
29. استخدم Anchors لو عندك أكثر من تجربة (Experiment) بتشترك في نفس القيم الأساسية.

```yaml
model:
  name: resnet50
  pretrained: true

training:
  batch_size: 32
  learning_rate: 0.001
  epochs: 50
  optimizer: adam

data:
  train_path: ./data/train
  val_path: ./data/val
```

مشروع ١٣: Logging Configuration

30. عرّف Formatters وHandlers وLoggers في ملف YAML منفصل عن الكود.
31. حمّله باستخدام logging.config.dictConfig بعد تحويله من YAML لـ Dictionary.

```yaml
version: 1
formatters:
  simple:
    format: "%(asctime)s - %(name)s - %(levelname)s - %(message)s"
handlers:
  console:
    class: logging.StreamHandler
    formatter: simple
```

```
root:
  level: INFO
  handlers: [console]
```

مشروع ١٤: CI/CD Pipeline مستقل (GitLab CI)

زي ما شفنا GitHub Actions في مشروع ١١، ده نفس الفكرة بأداة مختلفة (GitLab CI) عشان توضح إن المبدأ عام مش خاص بأداة واحدة:

32. عرّف مراحل الـ Pipeline (Stages): build, test, deploy.
33. عرّف Job لكل مرحلة وحدد فيه الأوامر المطلوبة.

```
stages:
  - build
  - test
  - deploy

build_job:
  stage: build
  script:
    - pip install -r requirements.txt

test_job:
  stage: test
  script:
    - pytest

deploy_job:
  stage: deploy
  script:
    - ./deploy.sh
  only:
    - main
```

مشروع ١٥: Python Project Configuration (mkdocs.yml)

34. عرّف عنوان المشروع والقائمة الجانبية (Navigation) لموقع توثيق مبني بـ MkDocs.

```
site_name: My Project Docs
theme:
  name: material
nav:
  - Home: index.md
  - Getting Started: getting-started.md
  - API Reference: api.md
```

مشروع ١٦: Application Settings شامل لتطبيق Backend

35. اجمع كل إعدادات التطبيق (Server, Database, Cache, Feature Flags) في ملف واحد منظم بأقسام واضحة.

```
app:
  name: MyService
```

```
  version: "2.3.0"

server:
  host: 0.0.0.0
  port: 8000

database:
  engine: postgresql
  pool_size: 10

features:
  new_dashboard: true
  beta_search: false
```

أفضل الممارسات

قسم الإعدادات لأقسام منطقية واضحة (app, server, database, features) بدل مفاتيح مسطحة مبعثرة، وده بيسهل قراءة وتوسيع الملف مستقبلًا.

أفضل الممارسات

أفضل الممارسات

ابدأ دايمًا بأبسط شكل ممكن للملف، وزود التعقيد تدريجيًا حسب الحاجة الفعلية، مش من أول لحظة.

ملخص الفصل

- المشاريع العملية دي بتغطي أشهر السيناريوهات الحقيقية في JSON: استجابات API موحّدة، تحويل بين الصيغ (CSV, Excel)، تخزين بيانات محلي، وفصل ملفات Settings.
- وفي YAML: Docker وKubernetes، أكتر من أداة CI/CD (GitHub Actions وGitLab CI)، إعدادات Machine Learning وLogging وMkDocs وApplication Settings الشاملة.
- كل مشروع اتبنى بنفس المنهجية: ابدأ بسيط، وزود التعقيد حسب الحاجة الفعلية.

تمارين عملية

تمرين ١: عدّل مشروع Docker Compose (مشروع ٩ في الفصل ده) عشان يضيف خدمة Nginx كـ Reverse Proxy أمام خدمة الـ API.

الحل المقترح

nginx: image: nginx:latest ports: ["80:80"] depends_on: [api]

تمرين ٢: ليه استخدام ملف JSON كقاعدة بيانات في مشروع ٧ غير مناسب لتطبيق فيه أكتر من مستخدم بيكتب في نفس الوقت؟

الحل المقترح

لأن الكتابة على ملف JSON مش Atomic بشكل افتراضي؛ لو مستخدمين اتنين قرأوا الملف في نفس اللحظة وعدّلوا وكتبوا، آخر واحد كتب هيمسح تعديلات التاني (Race Condition)، وده مش مشكلة موجودة في قواعد البيانات الحقيقية اللي بتدير القفل (Locking) والتزامن بشكل صحيح.

الفصل الثاني عشر — المقارنات

الفصل ده مرجع سريع يجمع كل المقارنات المهمة بين JSON وYAML وأشهر الصيغ البديلة، عشان تقدر تختار الصيغة الصح لأي موقف بسرعة.

JSON مقابل YAML

المعيار	JSON	YAML
سهولة القراءة البشرية	متوسطة	عالية جدًا
سهولة الكتابة اليدوية	متوسطة (أقواس كتيرة)	عالية
الأداء	أسرع بكتير	أبطأ نسبيًا
دعم التعليقات	❌	✅
أفضل استخدام	APIs وتبادل بيانات	ملفات إعداد وDevOps

الخلاصة: JSON للآلة، YAML للإنسان. لو الملف هيتقرأ آليًا بمعدل عالي، اختر JSON. لو الملف هيتعدّل يدويًا بانتظام، اختر YAML.

JSON مقابل XML

XML	JSON	المعيار
أكبر (Verbose)	أصغر بكتير	الحجم
أبطأ	أسرع	سرعة الـ Parsing
✅	❌	دعم Namespaces وAttributes
XSD (أقوى وأقدم)	JSON Schema	التحقق من الصحة (Validation)
أنظمة قديمة (Legacy) وSOAP	الأوسع انتشارًا	الاستخدام الحديث

XML لسه مستخدمة بقوة في أنظمة قديمة (Banking, SOAP Web Services) وفي المستندات المعقدة (زي Microsoft Office Formats)، لكن JSON هي الخيار الافتراضي لأي مشروع جديد.

JSON مقابل TOML

TOML	JSON	المعيار
✅	❌	دعم التعليقات
عالية جدًا	متوسطة	البساطة للملفات المسطحة
أقل وضوحًا مع العمق الكبير	عالية	البساطة للبيانات المتداخلة بعمق
ملفات إعداد (pyproject.toml, Cargo.toml)	APIs	الاستخدام الشائع

YAML مقابل TOML

TOML	YAML	المعيار
منخفضة (بنية أوضح بالأقواس)	عالية جدًا	الحساسية للإزاحة
أضعف نسبيًا	ممتازة	البيانات المتداخلة العميقة
عالية	متوسطة	البساطة للمبتدئين
أقل انتشارًا في هذا المجال	الأوسع (Kubernetes, CI/CD)	الانتشار في DevOps

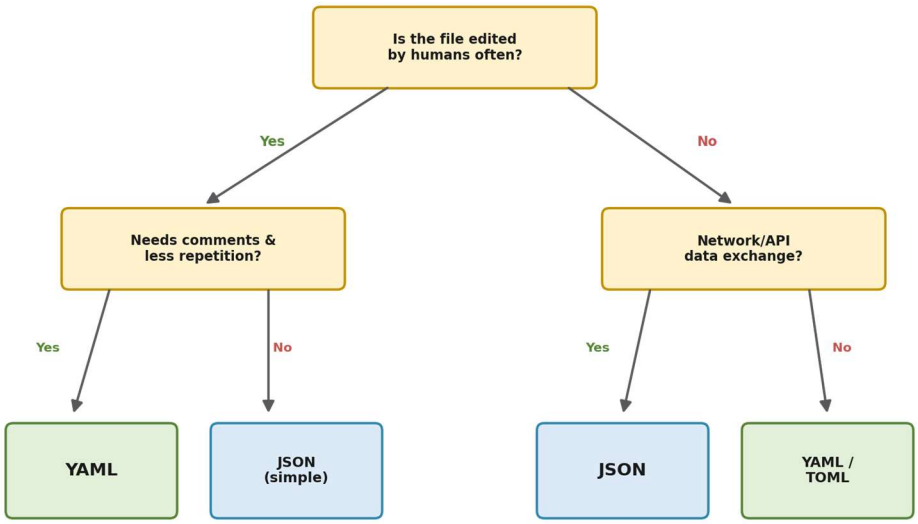
YAML مقابل INI

INI	YAML	المعيار
❌ محدودة جدًا (مستوى واحد أو اثنين)	✅ تدعم بعمق	البيانات المتداخلة
كل شيء نص أساسًا	غنية (نص، رقم، Boolean، قوائم)	أنواع البيانات
أنظمة قديمة (Windows Config)	واسع جدًا	الانتشار الحديث

YAML مقابل ملفات ENV

ملفات .env تخزن متغيرات بيئة بسيطة (KEY=VALUE) بدون أي تداخل أو أنواع بيانات معقدة، وتُستخدم عادةً لتمرير أسرار وإعدادات بسيطة للتطبيق وقت التشغيل. YAML أقوى بكتير في تمثيل هياكل معقدة، لكن ملفات .env أبسط وأسرع للإعدادات المسطحة القليلة، وبتتكامل بسهولة مع أدوات زي Docker وdotenv.

```
# .env
DATABASE_URL=postgresql://localhost:5432/app
DEBUG=false
```



شكل ١٢,١: شجرة قرار مبسطة لاختيار الصيغة المناسبة

جدول القرار السريع: أي صيغة أختار؟

الصيغة الموصى بها	الموقف
JSON	API Request/Response
YAML	ملف إعداد Kubernetes/Docker/CI-CD
TOML	إعداد مشروع Rust/Python بسيط (بدون تداخل عميق)
ENV Files	متغيرات بيئة بسيطة وقت التشغيل
XML	نظام قديم يعتمد على SOAP أو مستندات معقدة

الصيغة الموصى بها	الموقف
YAML	ملف إعداد كبير يحتاج تعليقات وتداخل عميق

الأسئلة الشائعة

❓ هل TOML هيستبدل YAML في المستقبل؟

لأ من غير المرجّح؛ TOML أبسط في الملفات المسطحة لكنها أضعف في تمثيل البيانات المتداخلة بعمق، وده بالظبط المجال اللي YAML قوية فيه (زي Kubernetes). كل صيغة لها مجالها المناسب، ومفيش مؤشر واضح إن TOML هتحل محل YAML في DevOps.

أخطاء شائعة

⚠ خطأ شائع

اختيار صيغة بناءً على "الموضة" أو "اللي الفريق اعتاد عليه" بدل تحليل طبيعة البيانات والاستخدام الفعلي.

أفضل الممارسات

☑ أفضل الممارسات

استخدم جدول القرار السريع فوق كنقطة انطلاق، لكن راجع قرارك دايمًا حسب سياق المشروع الفعلي ومتطلباته الخاصة.

ملخص الفصل

- لكل صيغة مجال قوة واضح: JSON لـ APIs، YAML لملفات الإعداد المعقدة، TOML للإعدادات المسطحة البسيطة، XML للأنظمة القديمة، ENV للمتغيرات البسيطة وقت التشغيل.
- القرار الصح مبني على تحليل الاستخدام الفعلي، مش على التفضيل الشخصي أو العادة.

تمارين عملية

تمرين ١: لو عندك مشروع Rust جديد ومحتاج ملف إعداد بسيط بدون تداخل عميق، إيه الصيغة الأنسب وليه؟

💡 الحل المقترح

TOML، لأنها المعيار الفعلي في مجتمع Rust (Cargo.toml) وأبسط بكتير للإعدادات المسطحة مقارنة بـ YAML أو JSON.

الفصل الثالث عشر — Professional Workflow

الفصل الأخير ده هيجمع كل حاجة اتعلمناها في إطار عملي واحد: إزاي مهندس Backend وMachine Learning وDevOps بيستخدموا JSON وYAML فعليًا في يومهم، مش بس نظريًا.

دورة حياة ملفات الإعداد (Configuration Lifecycle)

١. التصميم: تحديد الأقسام المطلوبة والقيم الافتراضية المناسبة لكل بيئة.

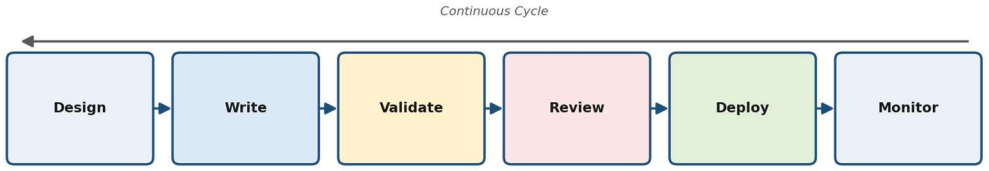
٢. الكتابة: كتابة الملف الأساسي (Base Config) مع فصل الأسرار من البداية.

٣. التحقق (Validation): تطبيق Schema أو Pydantic Model قبل قبول أي إعداد جديد.

٤. المراجعة (Code Review): أي تعديل في ملف إعداد حساس (خصوصًا Production) لازم يمر بمراجعة زي أي كود آخر.

٥. النشر (Deployment): تطبيق الإعداد في البيئة المستهدفة عبر أدوات آلية (CI/CD)، مش يدويًا.

٦. المراقبة والصيانة: متابعة أي مشاكل ناتجة عن إعداد خاطئ، وتحديث الملف بمرور الوقت مع توثيق كل تغيير جوهري.



شكل ١٣,١: دورة حياة ملف الإعداد كعملية مستمرة، مش خطوة تُنفَّذ مرة واحدة

تنظيم ملفات Configuration في المشاريع الكبيرة

أفضل هيكل مجلدات شائع في مشاريع Backend/ML كبيرة:

```
project/
├── config/
│   ├── base.yaml          # القيم المشتركة بين كل البيئات
│   ├── development.yaml
│   ├── testing.yaml
│   └── production.yaml
├── secrets/
│   └── .gitignore         # يستثني كل الأسرار الفعلية
├── k8s/
│   ├── deployment.yaml
│   └── service.yaml
├── .github/
│   └── workflows/
│       └── ci.yaml
└── src/
```

☑ أفضل الممارسات

افصل الإعدادات (Configuration) عن الكود بشكل كامل — أي قيمة ممكن تتغير بين البيئات المختلفة لازم تيجي من ملف إعداد أو متغير بيئة، مش مكتوبة داخل الكود نفسه (Hardcoded).

إدارة البيئات المختلفة (Development / Testing / Production)

النمط الشائع جدًا في الشركات الكبرى: ملف base.yaml فيه القيم المشتركة، وكل بيئة عندها ملف صغير بيتجاوز (Override) بس القيم المختلفة عن الأساس:

```
# base.yaml
logging:
  format: json
server:
  workers: 4

# production.yaml (وقت التشغيل بيتم دمجه فوق base.yaml)
server:
  workers: 16
database:
  host: prod-db.internal
```

الدمج (Merging) ده بيتم برمجيًا في الكود (بدمج الـ Dictionaries) أو عن طريق أدوات متخصصة زي Hydra في مجال الـ Machine Learning اللي مصممة خصيصًا لدمج ملفات إعداد متعددة المستويات.

إدارة الإعدادات الحساسة (Secrets Management)

- لا تخزّن الأسرار في نفس ملفات YAML/JSON العادية اللي بتتحفظ في Git.
- استخدم متغيرات البيئة (Environment Variables) للأسرار البسيطة، وحمّلها في الكود وقت التشغيل بدل تضمينها في الملف.
- في بيئات الإنتاج الكبيرة، استخدم أدوات متخصصة: AWS Secrets Manager، HashiCorp Vault، Kubernetes Secrets.
- Kubernetes نفسها بتفرّق بوضوح بين ConfigMap (إعدادات عادية غير حساسة) وSecret (بيانات حساسة، مع تشفير إضافي وصلاحيات وصول محدودة).

```
apiVersion: v1
kind: Secret
metadata:
  name: db-credentials
type: Opaque
data:
  password: cGFzc3dvcmQxMjM=    # Base64 encoded, وليس تشفير حقيقي
```

> **ملاحظة**
>
> مهم جدًا نوضح: الـ Base64 encoding في Kubernetes Secrets مش تشفير (Encryption) حقيقي، هو مجرد ترميز بسيط للنص. الحماية الفعلية بتيجي من صلاحيات الوصول (RBAC) والتشفير على مستوى التخزين (Encryption at Rest)، مش من الـ Base64 نفسها.

كيف يستخدم كل تخصص هذه الصيغ فعليًا

Backend Engineer

بيستخدم JSON لكل الـ APIs (استقبال وإرجاع البيانات)، وYAML لملفات إعداد التطبيق نفسه (اتصال قاعدة بيانات، إعدادات Logging، متغيرات البيئة الأساسية).

Machine Learning Engineer

بيستخدم YAML بكثافة لتعريف الـ Hyperparameters وإعدادات التجارب (Experiments)، وJSON لتخزين نتائج التجارب والـ Metrics اللي هتتقرأ آليًا بواسطة أدوات المراقبة (Dashboards).

DevOps Engineer

العمود الفقري لشغله بالكامل تقريبًا مبني على CI/CD، Docker Compose، YAML: Kubernetes Manifests، Ansible Playbooks، Pipelines. الأداء العالي هنا مش أولوية لأن الملفات دي بتتقرأ مرة أو مرات قليلة أثناء النشر، والأولوية هي الوضوح وسهولة الصيانة الجماعية.

الأسئلة الشائعة

❓ هل الشركات الكبرى بتستخدم أدوات جاهزة لدمج ملفات الإعداد ولا بتكتب الكود ده بنفسها؟

الاتنين موجودين؛ فيه أدوات جاهزة قوية (Hydra في ML، Helm في Kubernetes، Kustomize) بتوفر دمج وقوالب متقدمة جاهزة، لكن كتير من الشركات الأصغر أو المشاريع البسيطة بتكتب منطق دمج بسيط بنفسها لو احتياجاتها محدودة.

أخطاء شائعة

⚠ خطأ شائع

كتابة نفس الإعداد في كل بيئة من الصفر بدل استخدام ملف Base مشترك مع Override بسيط للفروقات.

⚠ خطأ شائع

الاعتقاد إن Base64 encoding في Kubernetes Secrets هو حماية كافية بحد ذاته.

أفضل الممارسات

☑ أفضل الممارسات

افصل الإعداد عن الكود دايمًا، افصل الأسرار عن الإعداد العام، واستخدم ملف Base مشترك مع Override لكل بيئة.

☑ أفضل الممارسات

اجعل أي تغيير في إعدادات Production يمر بمراجعة كود (Code Review) واضحة، بالظبط زي أي تغيير برمجي آخر.

ملخص الفصل

- دورة حياة أي ملف إعداد احترافي بتمر بمراحل: تصميم، كتابة، تحقق، مراجعة، نشر، ومراقبة مستمرة.
- الهيكل الشائع في المشاريع الكبيرة: ملف Base مشترك + ملفات Override لكل بيئة، مع فصل كامل للأسرار.
- كل تخصص (Backend, ML, DevOps) بيستخدم JSON وYAML بنسب وأغراض مختلفة حسب طبيعة شغله اليومي.

تمارين عملية

تمرين ١: اشرح بالتفصيل ليه الفصل بين ملف base.yaml وملفات override لكل بيئة أفضل من كتابة كل بيئة بشكل مستقل تمامًا.

> **الحل المقترح**
>
> لأن أي تغيير في قيمة مشتركة (زي صيغة الـ Logging) هيحتاج تعديل مكان واحد بس (base.yaml) بدل تعديل كل ملفات البيئات المنفصلة، وده بيقلل احتمال نسيان تحديث بيئة معينة ويحافظ على الاتساق بين كل البيئات.